



***Asociación Profesional del Cuerpo Superior
de Sistemas y Tecnologías de la Información
de la Administración del Estado***

**Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.**

TEMAS ESPECÍFICOS III: Ingeniería de los Sistemas de Información

**Tema 93. Técnicas de diseño de software. Diseño por capas y
patrones de diseño**

AUTOR: José Antonio Eusamio Mazagatos

Actualización 2017

ÍNDICE

1	INTRODUCCIÓN	4
1.1	DISEÑO ESTRUCTURADO	4
1.1.1	OBJETIVOS DEL DISEÑO ESTRUCTURADO.....	4
1.1.2	PRINCIPIOS GENERALES DEL DISEÑO ESTRUCTURADO	5
1.1.3	ARQUITECTURAS DE SISTEMAS MÁS COMUNES.....	5
1.1.4	TAREAS A REALIZAR EN LA FASE DE DISEÑO	6
1.2	DIAGRAMAS DE ESTRUCTURA DE CONSTANTINE.....	6
1.2.1	COMPONENTES DEL DIAGRAMA DE ESTRUCTURA DE CUADROS.....	6
1.2.2	EL PROCESO DEL DISEÑO ESTRUCTURADO	9
1.2.3	ANÁLISIS DE TRANSFORMACIÓN	9
1.2.4	ANÁLISIS DE TRANSACCIÓN.....	11
1.2.5	EJEMPLO DE DIAGRAMAS DE ESTRUCTURA DE CUADROS (TÉCNICAS MÉTRICA 3) ..	12
2	CRITERIOS DE EVALUACIÓN DE LA CALIDAD DEL DISEÑO	14
2.1	ACOPLAMIENTO	14
2.2	COHESIÓN	15
2.3	GUÍAS DE DISEÑO	17
2.3.1	DESCOMPOSICIÓN	17
2.3.2	FAN-OUT Y FAN-IN	17
3	DISEÑO ORIENTADO A OBJETOS (DOO)	18
3.1	DISEÑO PARTIENDO DE UN ANÁLISIS ORIENTADO A OBJETOS	18
3.2	DISEÑO PARTIENDO DE UN ANÁLISIS NO ORIENTADO A OBJETOS	18
3.3	CARACTERÍSTICAS DEL DISEÑO ORIENTADO A OBJETOS	19
3.3.1	MODELADO Y METODOLOGÍAS DE DISEÑO ORIENTADAS A OBJETOS.....	19
3.4	PRINCIPIOS OO.....	21
3.4.1	PRINCIPIOS SOLID	21
3.4.2	PRINCIPIOS GRASP.....	22
3.5	PROGRAMACIÓN ORIENTADA A ASPECTOS	23
3.5.1	EVOLUCIÓN HISTÓRICA	23
3.5.2	PRINCIPIO SEPARACIÓN DE INCUMBENCIAS	24
3.5.3	FUNDAMENTOS DE LA PROGRAMACIÓN ORIENTADA A ASPECTOS.....	25
3.6	DISEÑO POR CAPAS	26
3.6.1	ARQUITECTURA DE 2 CAPAS	27
3.6.2	MIDDLEWARE.....	27
3.6.3	SISTEMA DE 3 CAPAS O MULTICAPA.....	28
3.6.4	ARQUITECTURA CLIENTE - SERVIDOR.....	29
3.6.5	COMPONENTES DE UNA ARQUITECTURA CLIENTE - SERVIDOR	30
3.6.6	TIPOS DE ARQUITECTURA CLIENTE / SERVIDOR.....	32
3.7	PATRONES DE DISEÑO.....	35
3.7.1	ORIGEN DE LOS PATRONES DE DISEÑO	35
3.7.2	OBJETIVOS DE LOS PATRONES.....	36

3.7.3	CATEGORÍAS DE PATRONES	36
3.7.4	ESTRUCTURAS O PLANTILLAS DE PATRONES	37
3.7.5	RELACIÓN DE PRINCIPALES PATRONES GoF (GANG OF FOUR).....	37
3.7.6	ANTIPATRONES DE DISEÑO SOFTWARE	38
4	GLOSARIO	40

1 Introducción

Este tema se puede ver como una continuación del tema 88 relativo a las arquitecturas de diseño.

Este tema se centrará en el apartado del Diseño, distinguiendo entre los dos grandes tipos existentes como son el Diseño Orientado a Objetos y el Diseño Estructurado, las evoluciones a partir de éstas así como los aspectos relativos a la calidad del diseño.

A modo de referencia, las técnicas de diseño estarían cubiertas por la guía de técnicas y prácticas de métrica. Esta guía no cubre aspectos como los patrones o los modelos por capas (que está más relacionado con el estilo arquitectónico) que se desarrollarán a lo largo del tema.

1.1 Diseño Estructurado

La idea original del diseño estructurado fue presentada en la década de los '70, por Larry Constantine, y continuada posteriormente por otros autores, como Myers, Yourdon y Stevens.

El diseño estructurado de sistemas se ocupa de la identificación, selección y organización de los módulos y sus relaciones. Se decide qué componentes son necesarios, y la interconexión entre los mismos, para solucionar un problema bien especificado. Se comienza con la especificación resultante del proceso de análisis, se realiza una descomposición del sistema en módulos estructurados en jerarquías, con características tales que permitan la implementación de un sistema que no requiera elevados costes de mantenimiento.

1.1.1 Objetivos del Diseño Estructurado

Entre los principales objetivos del diseño estructurado se encuentran:

1. **Máxima inteligibilidad del sistema**, obteniendo la estructura modular y los detalles de proceso del sistema. Facilitar la comprensión de la aplicación redundante en la disminución del coste (tiempo/medios) asociado al aprendizaje del mismo desde el punto de vista técnico, y redundante en una mejora en los plazos y garantías de éxito en el mantenimiento del sistema.
2. **Minimizar el coste asociado al mantenimiento**.
La dificultad de mantenimiento, tanto correctivo como preventivo, debe reducirse para garantizar una correcta explotación del sistema. Ello incluye, por ejemplo, la reutilización en la medida de lo posible de código ya existente y probado.
3. **Facilitar la prueba**.
A medida que aumenta la complejidad del sistema se produce un incremento significativo en la complejidad de la prueba. Una adecuada estructuración modular persigue la simplificación de este proceso. Este objetivo se materializa en el desarrollo de procedimientos de prueba durante el diseño.
4. **Integración del sistema**.
Ello supone garantizar el correcto encaje del sistema como parte de un Sistema de Información Gerencial global, es decir, obtener una aplicación plenamente integrada con el resto de subsistemas de información de la organización.
5. **Que se garanticen los requisitos “no funcionales” de Reusabilidad, Fiabilidad y Portabilidad**
6. **Utilización de herramientas gráficas para la representación de la estructura modular del sistema** (en Análisis Estructurado se usará la técnica de Estructura de Cuadros de Constantine – DEC)

1.1.2 Principios Generales del Diseño Estructurado

Las técnicas de diseño estructurado se basan en una serie de principios generalmente aceptados para alcanzar los objetivos descritos en el punto anterior. Tales principios son:

- ✓ **Modularidad (Descomposición modular):** Se busca que el software esté dividido en componentes que se integran para satisfacer los requisitos del problema.
 - ✓ Es interesante que sean del menor tamaño posible.
 - ✓ Sistema más fácil de entender y modificar => para facilitar el mantenimiento.
 - ✓ Minimiza la duplicidad de código.
- ✓ **Jerarquía entre módulos:** de forma que los módulos de niveles superiores coordinen a los de niveles inferiores. Al ser una división jerárquica se puede controlar el número de módulos que interactúan con cualquiera de los otros.
- ✓ **Independencia modular:** Cada uno de los módulos del sistema debe cumplir una función bien definida e independiente del resto. Esto permite aislar errores que se producen en un módulo sin afectar a los demás, o hacer modificaciones en módulos de manera aislada
- ✓ **Modelización conceptual:** La estructuración del sistema en módulos no debe atender a un criterio exclusivamente técnico sino que debe reflejar conceptualmente la estructura de la organización a la que sirve (por ejemplo, para facilitar su comprensión).
- ✓ **Principio de “caja negra”:** Una correcta definición de las interfaces internas (entre módulos) y externas (entre subsistemas) permitirá ignorar la estructura interna del módulo tratado y centrar la atención en los resultados de la función que realiza.

1.1.3 Arquitecturas de Sistemas más Comunes.

La mayoría de los problemas se pueden resolver utilizando alguna de las arquitecturas que mencionaremos a continuación, o bien una combinación de ellas.

- ✓ **Transformación por lotes:** Una transformación por lotes o “batch” es una transformación secuencial de una entrada en una salida sin ningún tipo de interacción con el exterior.
- ✓ **Transformación continua:** Una transformación continua es un sistema en el que las salidas dependen activamente de entradas cambiantes y deben ser periódicamente actualizadas.
- ✓ **Interfaz interactiva:** Este tipo de sistemas está dominado por la interacción entre el sistema y agentes exteriores independientes de éste.
- ✓ **Simulación dinámica:** Estos sistemas modelan objetos, materiales, sistemas, comportamientos, etc., del mundo real.
- ✓ **Sistema en tiempo real:** Un sistema en tiempo real es un sistema interactivo con fuertes restricciones de tiempo. El sistema debe garantizar la respuesta dentro de un intervalo de tiempo cerrado (no necesariamente corto, pero sí estricto en su cumplimiento). Por ejemplo 25 imágenes por segundo, Tiempo de respuesta Inferior a “X” msg.
Estos sistemas son complejos, conllevan un control exhaustivo de las interrupciones y la coordinación de las distintas C.P.U's.
- ✓ **Controlador de transacciones:** Un controlador de transacciones es una base de datos cuya función principal es almacenar y acceder a información (Monitor Transaccional).
La mayoría de estos sistemas transaccionales trabajan con múltiples usuarios y utilizan concurrencia. Cada transacción debe ser manejada como una entidad sin relación con las otras transacciones.

Los sistemas en tiempo real y los sistemas transaccionales se analizan en profundidad en el tema 088 del temario al cual remitimos.

Es de resaltar que en un sistema completo y complejo tendremos una mezcla de estilos diferentes para cada parte del sistema.

1.1.4 Tareas a realizar en la fase de diseño

Entre las tareas a realizar en la fase de diseño del sistema de información se encuentran, reconocidas por la mayoría de autores de Ingeniería del Software:

Relativas al diseño de “alto nivel”:

- ✓ **Selección de la arquitectura global del sistema.** O bien viene impuesta por el Plan de Sistemas donde esté encuadrado el proyecto o porque hay similitud con soluciones ya adoptadas.
- ✓ **Descomposición en subsistemas de sistemas.** Tratamos aquí subsistemas físicos y no lógicos como en la fase de análisis.
- ✓ **Identificación de concurrencias.** Se trata de buscar un compromiso que atienda a las limitaciones del hardware que estamos considerando, al rendimiento que esperamos para nuestras aplicaciones y al coste del sistema en general.
- ✓ **Control de los recursos compartidos.** Hay que considerar controles sobre el acceso al hardware y sobre el acceso a las bases de datos (p.e.). Todos los recursos globales del sistema tienen que gestionarse de forma que se garantice el acceso a los mismos por todas aquellas entidades autorizadas.
- ✓ **Gestión de los datos.** Elección del diseño físico más adecuado para almacenar los datos del sistema.
- ✓ **Diseño de las interfaces.** Entre los distintos componentes que forman el sistema (sw-hw, sw-sw, etc).

Relativas al diseño de “bajo nivel”:

- ✓ **Identificación y manejo de excepciones.** En la fase de análisis se responde a lo que se quiere que el sistema haga. Aquí se pretende completar el análisis y describir lo que el sistema no tienen que hacer y también qué acciones hay que tomar cuando se presenta una excepción.
- ✓ **Selección de los algoritmos y de las estructuras de datos.** Aquí nos referimos al diseño procedimental o diseño de bajo nivel.
- ✓ **Preparación para la migración y carga inicial de datos.**
- ✓ **Especificación técnica de las pruebas.**

1.2 Diagramas de Estructura de Constantine

Fueron presentados en la década de los 70 como la principal herramienta utilizada en diseños estructurados, por autores como Constantine, Myers, Stevens e Yourdon.

Los diagramas de estructura son una herramienta para modelar los módulos de un sistema y sus relaciones y, junto con las especificaciones de funcionalidad de los módulos y las estructuras de datos, componen un diseño inicial que deberá ser analizado y mejorado. Permiten modelar un programa como una jerarquía de módulos. Cada nivel de la jerarquía representa una descomposición más detallada del módulo del nivel superior.

En el diseño estructurado orientado al flujo de datos, partimos de la representación del flujo de la información obtenida en la fase de análisis, donde la información puede representarse como un flujo continuo que sufre una serie de transformaciones conforme va de la entrada a la salida.

Para su realización se partirá del modelo de procesos obtenido como resultado de la aplicación de la técnica de diagrama de flujo de datos (DFD).

1.2.1 Componentes del Diagrama de Estructura de Cuadros

La notación usada se compone básicamente de tres símbolos: módulos, invocaciones (relaciones entre módulos) y parámetros de entrada o valores retornados (comunicaciones entre módulos). Existen otros componentes secundarios, almacenes y dispositivos.

1.2.1.1 Módulo

Un módulo es una parte lógica separable de un programa (división de software clara y manejable con interfaces modulares perfectamente definidas). Una parte del código que puede ser llamada para desempeñar una función.

Puede representar un programa, subprograma o rutina dependiendo del lenguaje a utilizar. Admite parámetros de llamada y retorna algún valor si es preciso.

Características:

- Deben ser de pequeño tamaño.
- Deben ser lo más independientes posible.
- Deben verse como cajas negras: realizarán una función clara y sencilla.

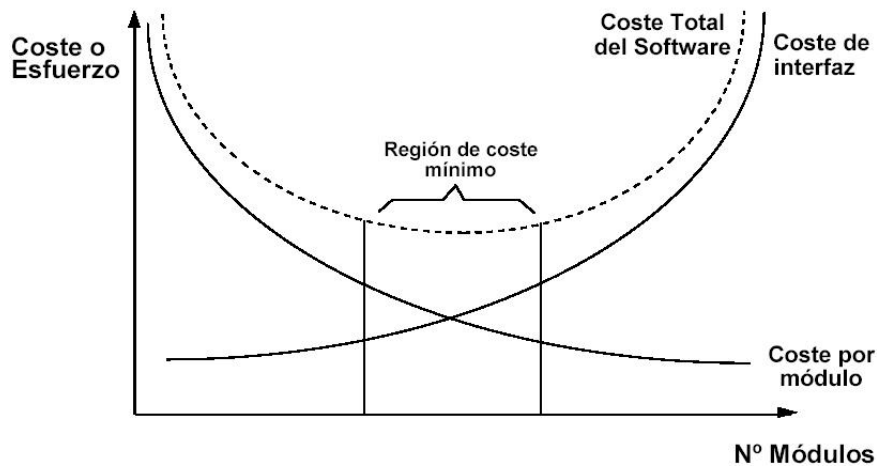


Figura 1 .- Coste de desarrollar sistemas SW en función del tamaño del módulo

El motivo por el cual el módulo debe ser de tamaño pequeño se entiende por la figura anterior, donde se representa el coste de desarrollar sistemas SW en función del tamaño del módulo. El coste por módulo disminuye si se aumenta el número de módulos, pero crece el coste de implementar interfaces. La curva de coste total del software tiene un mínimo en el punto medio, denominada región de coste mínimo hacia el que se debería tender. Es una "medida" eurística, basada en la experiencia, no una métrica precisa.

Tipos de módulos:

- Predefinidos: aquellos módulos estándar o ya definidos por el lenguaje (lectura de datos por librería, escritura en impresora,...). Llevan dos rayas verticales a los lados.
- Normales: aquellos que surgen del estudio de la estructura modular del sistema.



Figura 2 .- Representación de un módulo normal y otro predefinido

1.2.1.2 Conexión

Representa una llamada de un módulo a otro. Para llamadas a módulos estáticos se utiliza trazo continuo y para llamadas a módulos dinámicos trazo discontinuo.

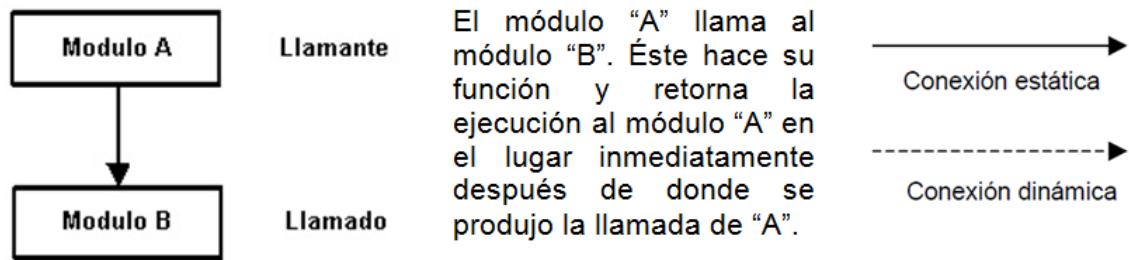


Figura 3 .- Invocación de un módulo llamante a otro llamado

Tipos de llamadas entre módulos:

- Secuencial: un módulo llama a otros una sola vez y se ejecutan de izquierda a derecha y de arriba abajo.
- Repetitiva - Iteración: cada uno de los módulos inferiores se ejecuta varias veces mientras se cumpla una condición. Se representa con una flecha abarcando los módulos cubiertos por la iteración.
- Alternativa - Decisión: cuando el módulo superior, en función de una decisión, llama a un módulo u otro de los de nivel inferior. Se representa con un rombo.

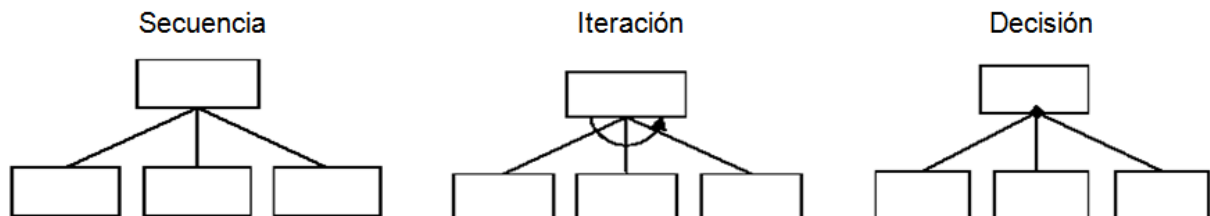
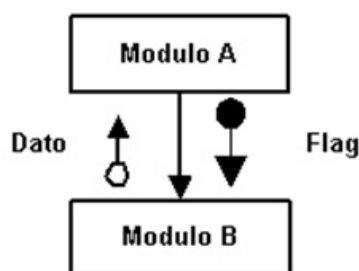


Figura 4 .- Tipos de llamadas entre módulos, Secuencial, Iteración, y Decisión

1.2.1.3 Parámetro

Representa la Información que se intercambia entre los módulos. Los parámetros son los datos de entrada. Pueden tener valores de retorno, que representan los datos devueltos por el módulo llamado y se denominan "resultado".



Tipos de parámetros:

- ✓ Control: son valores de condición que afectan a la lógica de los módulos llamados. Sincronizan la operativa de los módulos.
- ✓ Datos: información compartida entre módulos y que es procesada en los módulos llamados.

Figura 5 .- Módulo Llamante A intercambia parámetros de control o flag en la petición

Se pueden representar también parámetros híbridos, de control y de datos, aunque en estos se suele referir a ellos como de "control", ya que sus implicaciones a nivel de calidad del diseño es mayor.

1.2.1.4 Otros componentes Almacenes y Dispositivos físicos

Otros componentes que se pueden representar en el diagrama de estructura son:



Almacén de datos. Es la representación física del lugar donde están almacenados los datos del sistema.

Dispositivo físico. Es cualquier dispositivo por el cual se puede recibir o enviar información que necesite el sistema.

Figura 6.- Almacenes y Dispositivos físicos en un Diagrama de Estructura (DEC)

1.2.2 El proceso del diseño estructurado

El Diseño Orientado al Flujo de Datos permite una cómoda transformación de las representaciones de la información (DFD) a una descripción de la estructura del programa. Este proceso debe seguir los siguientes pasos:

1. Establecer el tipo de flujo de información:
 - a. Flujo de transformación: flujo de entrada, transformación, flujo de salida.
 - b. Flujo de transacción: flujo de datos de entrada, evaluación de la transacción, selección del camino de acción a ejecutar en función de la evaluación.
2. Determinar los límites del flujo.
3. Convertir el DFD en la estructura del programa, convirtiendo las transformaciones (burbujas del DFD) en módulos. El tipo de flujo de información del paso 1 es lo que determina el método de conversión requerido en el paso 3.
4. Definir la jerarquía de control descomponiéndola mediante particionamiento. Factorizar la estructura.
5. Refinar la estructura resultante usando medidas y heurísticas de diseño.

En los DFD de un sistema, generalmente estarán presentes los dos tipos de flujos: el flujo de transformación y el flujo de transacción. El diseño orientado al flujo de datos comienza con una evaluación del DFD a nivel 2 ó 3. Se examinan los DFD hasta el nivel en que la transformación tiene una cohesión alta (cada transformación ejecuta una función sencilla y diferenciada) pudiéndose implementar como un módulo. A partir de aquí se determinará si el DFD tiene características de transformación o transacción.

1.2.3 Análisis de transformación

Es un conjunto de pasos de diseño que permite obtener, a partir de un DFD con características de transformación, la estructura de diseño de alto nivel del sistema. Un **DFD con características de transformación, como se ve en la siguiente figura**, es aquél en el que se pueden distinguir:

- ✓ Flujo de llegada o entrada (Obtener datos de entrada).
- ✓ Flujo de transformación o **centro de transformación** que contiene los procesos esenciales del sistema y es independiente de las características particulares de la entrada y la salida.
- ✓ Flujo de salida (Producir datos de salida).

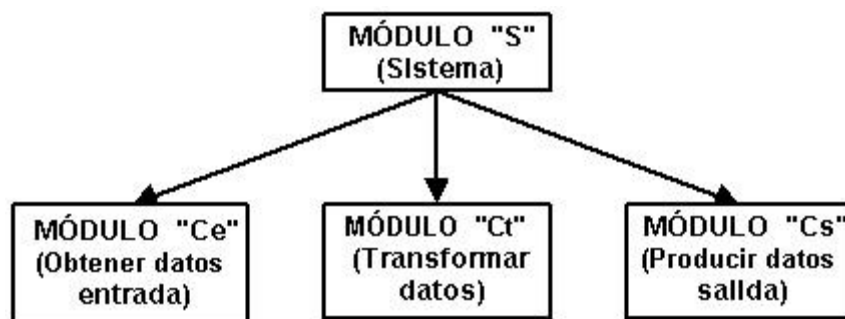


Figura 7.- Diagrama de Cuadros con características de transformación

Los datos que necesita el sistema se recogen por los módulos que se encuentran en las ramas de la izquierda, de modo que los datos que se intercambian en esa rama serán ascendentes. En las ramas

centrales habrá movimiento de información compartida, tanto ascendente como descendente. En las ramas de la derecha, la información será de salida y, por lo tanto, descendente.

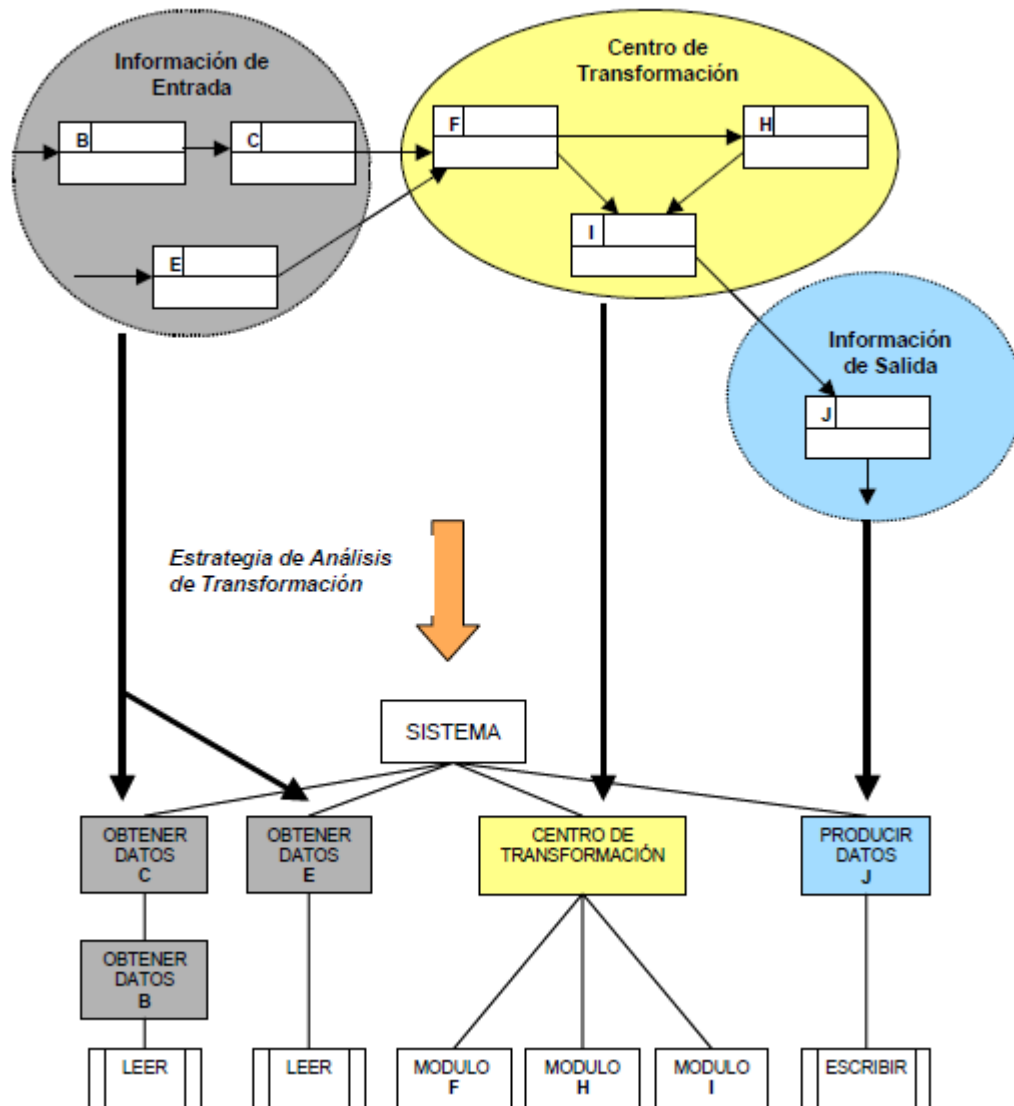


Figura 8 .-DFD con características de transformación y DEC asociado (ejemplo de guía de técnicas de Métrica)

Los **pasos a realizar en el análisis de transformación** son:

1. Identificar el centro de transformación. Para ello será necesario delimitar los flujos de llegada y salida de la parte del DFD que contiene las funciones esenciales del sistema.
2. Realizar el **“primer nivel de factorización”** o descomposición del diagrama de estructura. Habrá que identificar tres módulos subordinados a un módulo de control del sistema:
 - Módulo controlador del proceso de información de entrada.
 - Módulo controlador del centro de transformación.
 - Módulo controlador del proceso de la información de salida.
3. Elaborar el **“segundo nivel de factorización”**. Se transforma cada proceso del DFD en un módulo del diagrama de estructura.
4. Revisar y factorizar (p.e. algún módulo controlador puede no ser necesario) la estructura del sistema utilizando medidas y guías de diseño.

1.2.4 Análisis de Transacción

El análisis de transacción se aplica cuando en un DFD existe un proceso que, en función del flujo de llegada, determina la elección de uno o más flujos de información. Se denomina **centro de transacción** al proceso desde el que parten los posibles caminos de información.

Los **pasos a realizar en el análisis de transacción** son:

1. Identificar el centro de transacción. Se delimita la parte del DFD en la que a partir de un camino de llegada se establecen varios caminos de acción.
2. Transformar el DFD en la estructura adecuada al proceso de transacciones. El flujo de transacciones se convierte en una estructura de programa con una bifurcación de entrada y una de salida.
3. Factorizar la estructura de cada camino de acción. Cada camino se convierte en una estructura que se corresponde con las características específicas del flujo (de transacción o de transformación).
4. Refinar la estructura del sistema utilizando medidas y guías de diseño.

La estructura que tendría el Diagrama de Estructura de Cuadros sería la que recoge la figura siguiente.

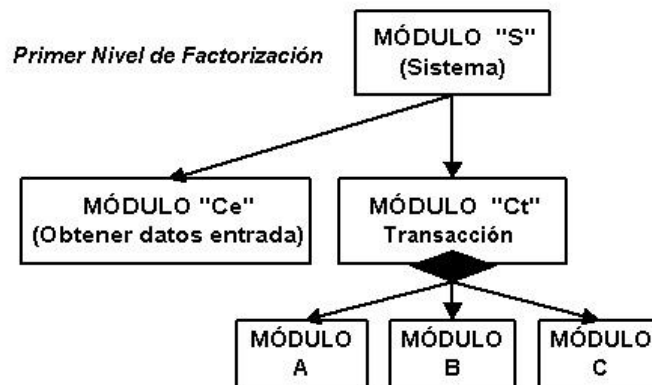


Figura 9 .- Diagrama de Cuadros con características de transacción

El proceso descrito anteriormente, se puede reflejar por completo desde un DFD con características de Transacción hasta un DEC con características de transacción como se recoge en la figura siguiente. Este tipo de diagramas y técnicas son de uso adecuado en el caso de diseño de sistemas transaccionales como se planteó en el tema 88.

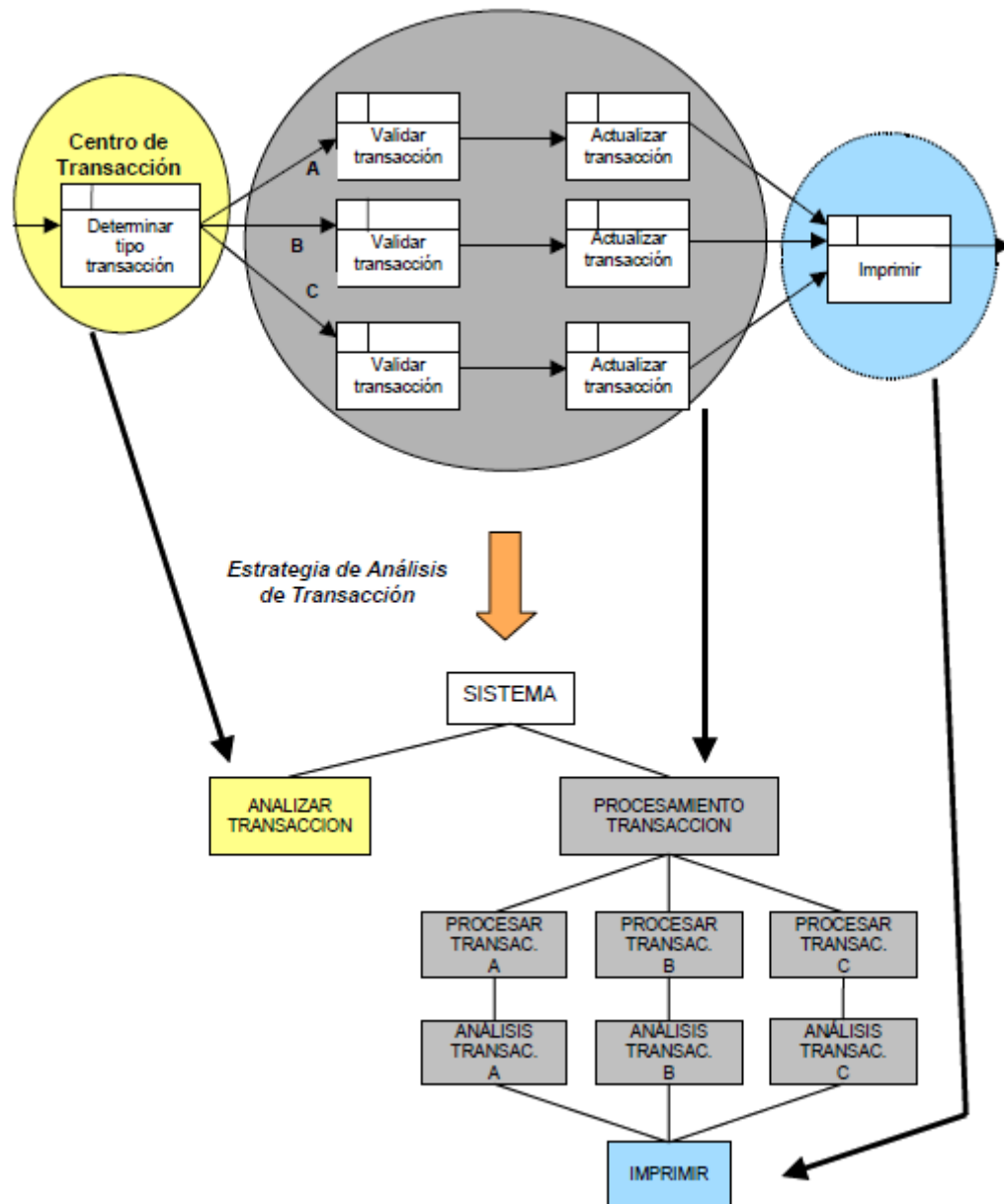


Figura 10 .-DFD con características de transacción y DEC asociado (ejemplo de guía de técnicas de Métrica)

La figura representa un DFD con características de transacción y DEC asociado. Un proceso "Determinar Tipo Transacción" es el que analiza la transacción a realizar transformándose en el módulo de control de Entrada "Analizar Transacción", y los procesos de validar transacción y actualizar transacción según 3 tipos, A, B y C se transforman en módulos independientes de ejecución en paralelo, para finalmente converger en el modulo Imprimir

1.2.5 Ejemplo de diagramas de Estructura de Cuadros (Técnicas Métrica 3)

El siguiente ejemplo muestra un DEC con características de transformación, proceso de emisión de cheques para el pago de nóminas de los empleados de una empresa. En él se diferencian los cálculos relativos a los trabajadores empleados por horas y los que poseen contrato. La lectura del fichero de empleados y la impresión de los cheques son módulos ya disponibles en las librerías del sistema, es decir, módulos predefinidos.

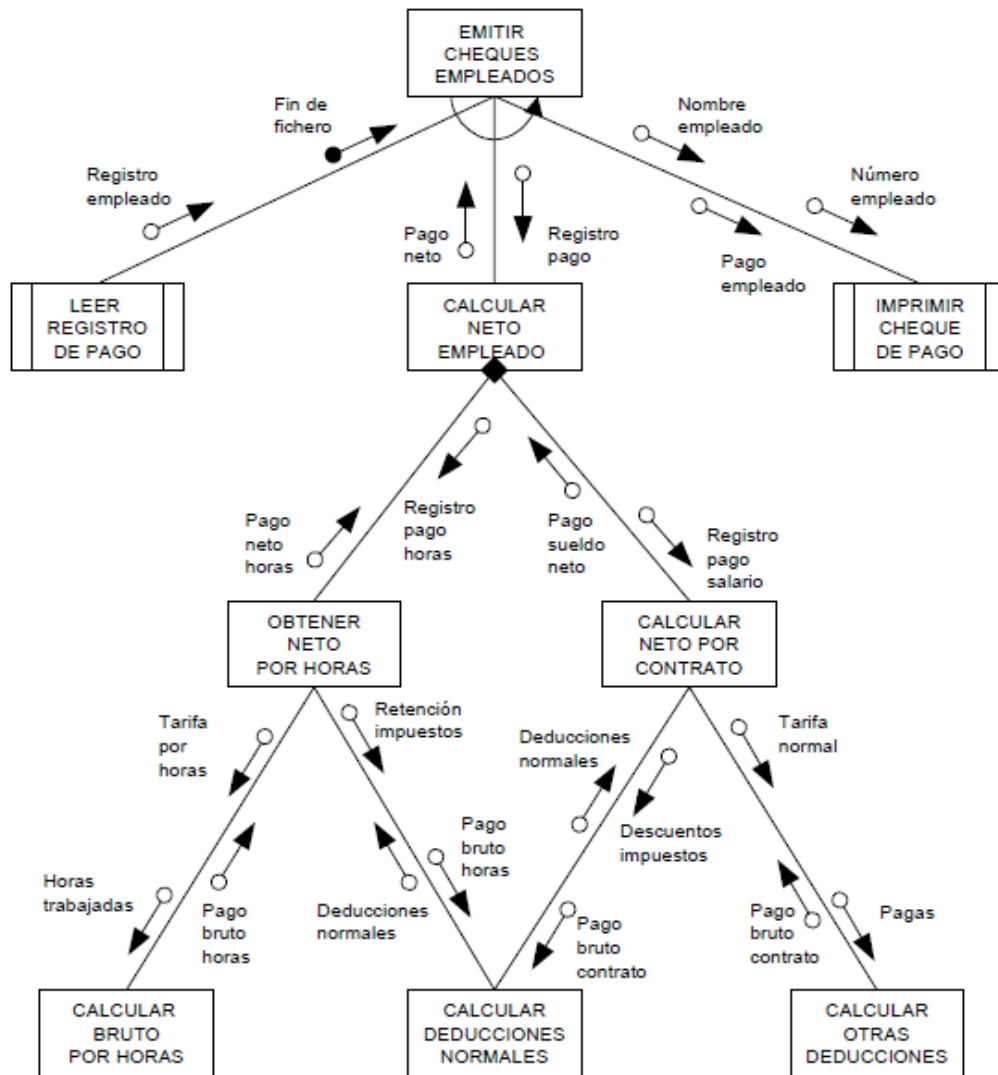


Figura 11 .- Ejemplo de Diagrama de Cuadros con características de transformación

El siguiente ejemplo muestra un DEC con características de transacción, en el que el módulo gestionar piscina invoca a la izquierda al módulo predefinido leer carnet, y luego invoca al módulo gestionar tipo entrada, que en función del carnet, decidirá invocar al módulo gestionar estudiante o al módulo gestionar Trabajador.

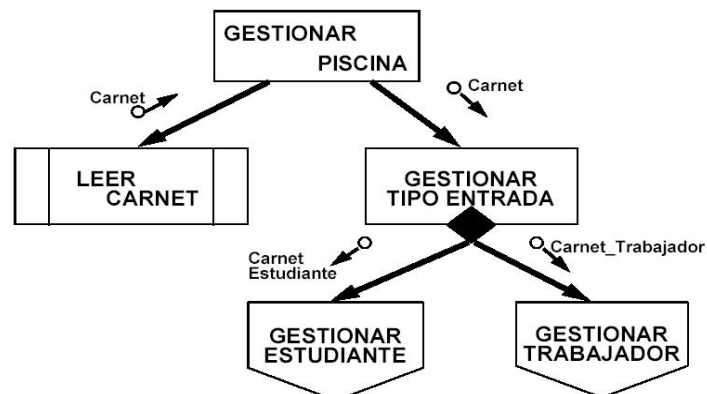


Figura 12 .- Ejemplo de Diagrama de Cuadros con características de transacción

2 Criterios de Evaluación de la Calidad del Diseño

Un buen diseño debe organizar la complejidad del problema de manera que el esfuerzo asociado con su desarrollo, prueba, entendimiento y mantenimiento pueda ser controlado y minimizado. En relación con esto, uno de los principios fundamentales del diseño estructurado es que el sistema debe ser dividido en una serie de módulos que resulten fácilmente manejables y la división se ha de llevar a cabo de forma que los módulos sean tan independientes como sea posible y que cada módulo sólo realice una función simple relacionada con el problema.

Se hace necesario, pues, evaluar la calidad de la estructura de los diseños software y, a estos efectos, se utilizan dos unidades de medida: el acoplamiento y la cohesión.

El Objetivo siempre debe ser conseguir un Acoplamiento Bajo y una Cohesión Alta.

2.1 Acoplamiento

Se define **acoplamiento** como el *grado de interdependencia entre los módulos*. Para un buen diseño, se debe intentar minimizar el acoplamiento, es decir, hacer que los módulos sean tan independientes unos de otros como sea posible.

Un bajo acoplamiento indica que se ha hecho una buena descomposición, y es deseable porque:

- ✓ Cuantas menos conexiones existan entre dos módulos, menos oportunidad habrá para que se produzca el “efecto onda”, es decir, que un defecto en un módulo pueda afectar a otro.
- ✓ Posibilita que cada cambio realizado en un módulo afecte lo menos posible a otros, esto es, permite cambiar un módulo con el mínimo riesgo de tener que cambiar otro.
- ✓ Garantiza que mientras se está manteniendo un módulo no hay necesidad de preocuparse de los detalles internos (código) de cualquier otro módulo.

De mejor a peor, se contemplan los siguientes **niveles de acoplamiento** recogidos en la siguiente figura:

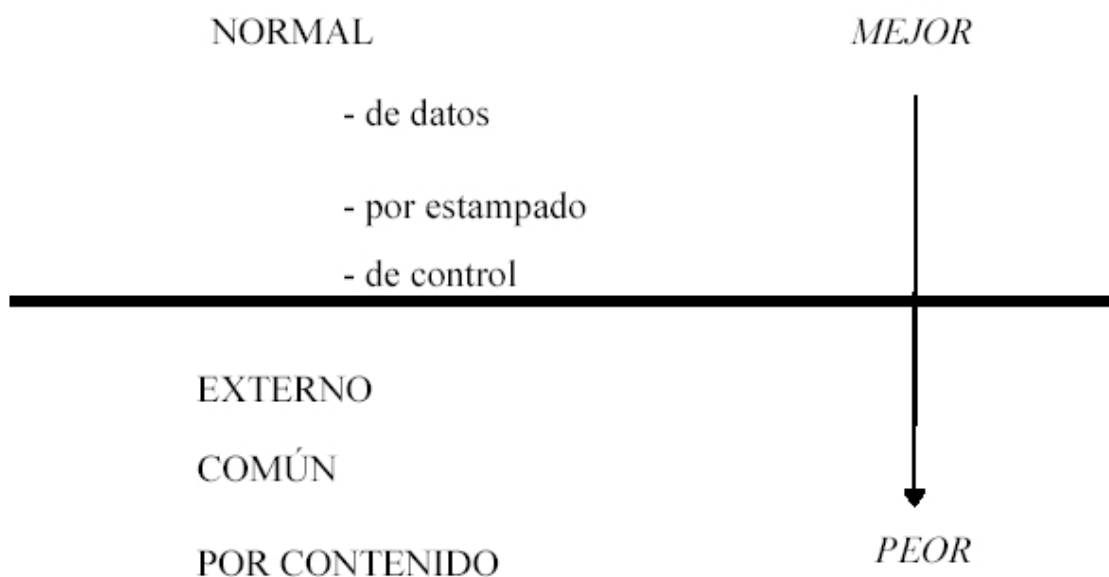


Figura 13 .- Tipos de Acoplamiento ordenados de Menor a Mayor

El **acoplamiento normal** es el que sucede cuando dos módulos intercambian datos, pero éstos no interfieren en la operativa normal de la función que realiza el módulo de nivel inferior. Es decir, en el acoplamiento normal un módulo "A" llama a otro "B", el módulo "B" se ejecuta y retorna el control al módulo "A".

En el acoplamiento normal, dependiendo de los datos que intercambien los módulos en su comunicación se distinguen tres tipos:

- ✓ **Acoplamiento de datos.** Los módulos se comunican mediante parámetros, donde cada parámetro es una unidad elemental de datos. Es el tipo de acoplamiento más deseable, salvo el normal, propiamente dicho, donde los módulos no intercambian ningún parámetro entre ellos.
- ✓ **Acoplamiento por estampado.** En la comunicación, los módulos se pasan datos con estructura de registro. Este tipo de acoplamiento no es muy deseable si el módulo que recibe el registro no necesita todos los elementos de datos que se le pasan, sino parte de ellos.
- ✓ **Acoplamiento de control.** Sucede si los datos que se intercambian en la comunicación entre módulos son controles. No es un buen acoplamiento porque no permite que los módulos sean totalmente independientes, debido a que un módulo controla a otro influyendo en su ejecución.

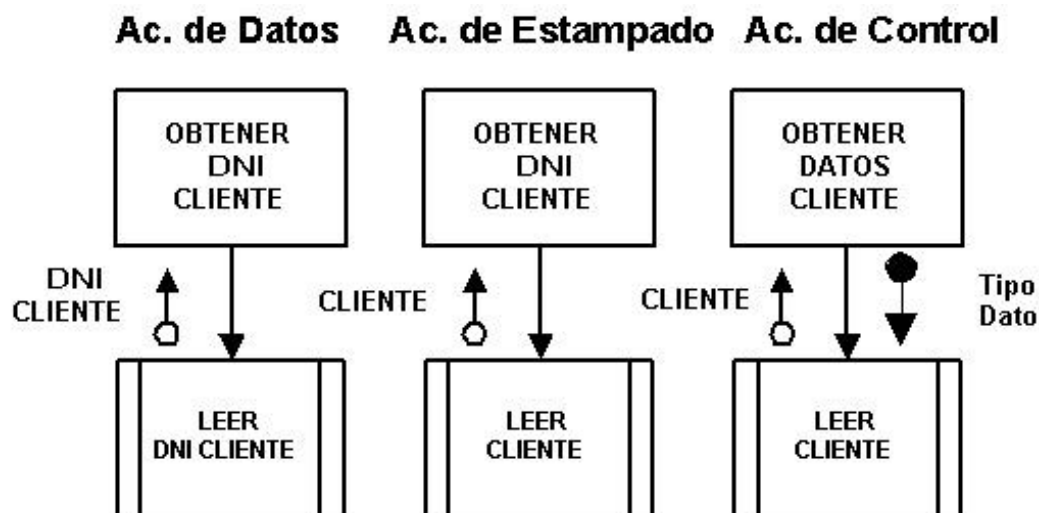


Figura 14 .- Tipo de acoplamiento "normal" según el parámetro intercambiado

- ✓ **El acoplamiento Externo:** los módulos están ligados a componentes externos (dispositivos E/S, protocolos de comunicaciones, etc.).
- ✓ **El acoplamiento común** es el que sucede cuando un número indeterminado de módulos (más de dos) hacen referencia a un área común de datos. Este tipo de acoplamiento es desaconsejable porque un defecto en un módulo que utiliza el área común puede transmitir ese error a otro módulo que utilice dicho área y porque los módulos así acoplados se modifican peor que los acoplados normalmente.
- ✓ El **acoplamiento por contenido** se presenta cuando un módulo hace referencia a la parte interior de otro, ya sea porque le modifica algún elemento, porque usa una variable local de ese módulo, porque ambos módulos compartan los mismos contenidos, etc. En cualquier caso, este tipo de acoplamiento es totalmente inaceptable porque rompe la jerarquía de funcionalidad de la estructura.

2.2 Cohesión

La cohesión hace referencia a la relación existente entre los elementos de un mismo módulo. Su objetivo es organizar los elementos de tal manera que los que tengan más relación a la hora de realizar una tarea pertenezcan al mismo módulo, y los elementos no relacionados figuren en módulos separados.

La **cohesión** se puede definir como la **medida de la relación funcional de los elementos de un módulo**; entendiendo por elementos, tanto la sentencia o grupo de sentencias que lo componen, como las definiciones de datos o las llamadas a otros módulos. Idealmente, un módulo coherente sólo debe hacer una única cosa.

El concepto de cohesión es importante a la hora del diseño, ya que cuanto mayor cohesión tengan los módulos, es menor el coste de programar y es mayor la calidad del sistema.

Ordenados de mayor a menor, se distinguen los siguientes tipos de cohesión:

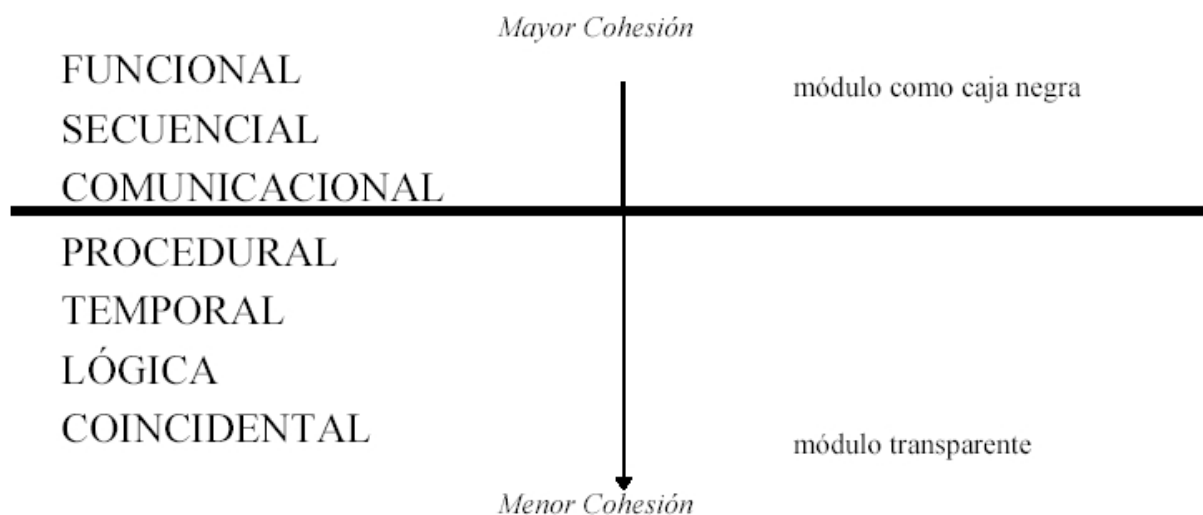


Figura 15 .- Tipos de cohesión ordenados de Mayor a Menor

- ✓ **Cohesión funcional.** Es el grado más alto de cohesión. Todos los elementos que componen el módulo están relacionados en el desarrollo de una única función.
- ✓ **Cohesión secuencial.** Un módulo realiza distintas tareas dentro de él en secuencia, de forma que las entradas de cada tarea son las salidas de la anterior.
- ✓ **Cohesión comunicacional.** Un módulo realiza actividades paralelas usando los mismos datos de entrada y salida.
- ✓ **Cohesión procedimental.** El módulo tiene una serie de funciones relacionadas por un procedimiento efectuado por el código. Es decir, todos los elementos de procesamiento de un módulo están relacionados y deben ejecutarse en un orden determinado. Existe paso de controles.
- ✓ **Cohesión temporal.** Un módulo presenta este tipo de cohesión cuando sus elementos están implicados en actividades relacionadas con el tiempo.
- ✓ **Cohesión lógica.** Se produce cuando las actividades que realiza el módulo tienen la misma categoría, es decir, es como si se tuvieran partes independientes dentro del mismo módulo.
- ✓ **Cohesión coincidente o casual.** Un módulo posee este tipo de cohesión cuando sus elementos contribuyen a las actividades relacionándose mutuamente de una manera poco significativa. Este tipo de cohesión viola el principio de independencia y de caja negra de los módulos, ya que es necesario que el módulo "padre" tenga conocimiento de la estructura interna de los módulos "hijos".

En caso de tener "cohesión causal" se recomienda encarecidamente separar en varios módulos y reevaluar la calidad interna del mismo, tanto en relación al acoplamiento (varios módulos) como a la cohesión (cada uno de los nuevos módulos resultantes).

El siguiente cuadro resume los distintos tipos de cohesión y como se debe elegir entre ellos:

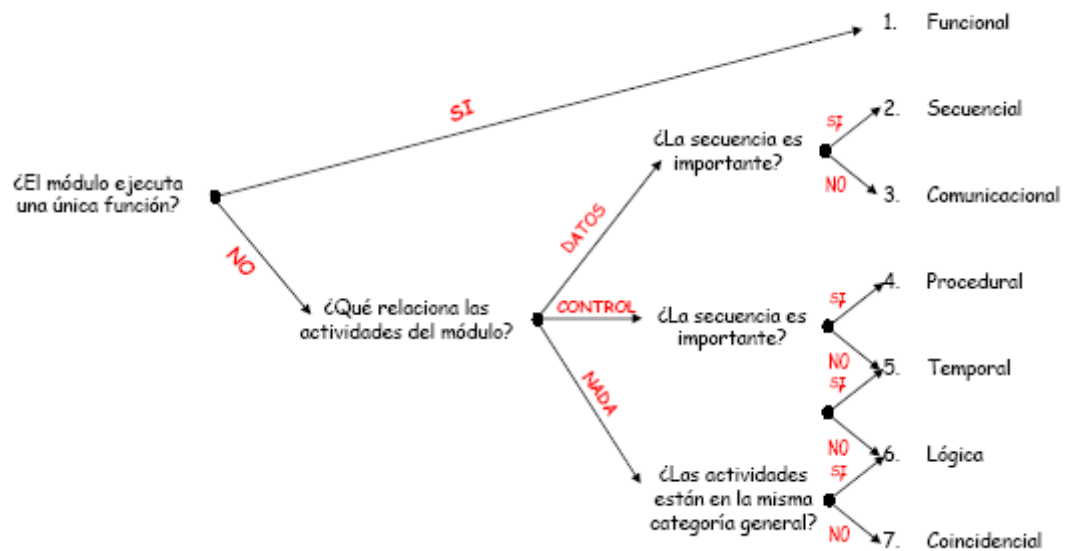


Figura 16 .- Como saber qué tipo de cohesión tiene el diseño Software

Un buen diseño debe ir orientado a conseguir que los módulos realicen una función sencilla e independiente de las demás (máxima cohesión) y que la dependencia con otros módulos sea mínima (acoplamiento mínimo), lo cual facilita el mantenimiento del diseño.

2.3 Guías de diseño

2.3.1 Descomposición

Se trata de separar una función contenida en un módulo, para que conforme un nuevo módulo. Puede hacerse por distintas razones: para reducir el tamaño del módulo que hará más sencillo el trabajar con él, por hacer el sistema más claro permitiendo una mejor comprensión del diseño al subdividir módulos, para minimizar la duplicación de código reutilizando éste en otras partes del sistema, o bien para crear módulos más generales que serán más útiles y reutilizables en el mismo sistema y, además, pueden ser generadas bibliotecas de módulos reutilizables en otros sistemas.

2.3.2 Fan-Out y Fan-In

Se define el *fan-out* de un módulo al número de subordinados inmediatos de un módulo (cantidad de módulos invocados). Si un módulo tiene un fan-out muy grande, entonces compone el trabajo de muchos módulos subordinados, lo que supone una mayor complejidad. Esto puede implicar una mayor propagación de errores. El Fan-out es usado como una medida de *complejidad*. Para tener acotada ésta se debe limitar el *fan-out* a no más de siete más o menos dos (7 ± 2). Un módulo con muchos subordinados puede fácilmente ser mejorado por descomposición.

El *fan-in* de un módulo es el número de superiores inmediatos de un módulo (la cantidad de módulos que lo invocan). Es usado como una medida de *reusabilidad*. Un alto fan-in es el resultado de una buena descomposición.

Dos características fundamentales deben ser garantizadas en módulos con un alto fan-in:

- ✓ **Buena Cohesión:** Los módulos con mucho fan-in deben tener alta cohesión, con lo cual es muy probable que tengan buen acoplamiento con sus llamadores;
- ✓ **Interfaz Consistente:** Cada invocación para el mismo módulo debe tener el mismo número y tipo de parámetros.

3 Diseño Orientado a Objetos (DOO)

En este apartado se intentará dar a conocer y describir los conceptos y aspectos fundamentales del diseño orientado a objetos (DOO) dentro del desarrollo de un producto software, así como las técnicas, metodologías y herramientas actuales de dicho paradigma en la Ingeniería de software.

Según [Booch \(1986\)](#), el Diseño Orientado a Objetos "es un método de diseño abarcando el proceso de descomposición orientado a objetos y una notación para representar ambos modelos lógico y físico tal como los modelos estáticos y dinámicos del sistema bajo diseño". Es un diseño de sistemas que utiliza objetos y clases de objetos.

El diseño orientado a objeto parte de un análisis cuyos requisitos y funcionalidades se han de cumplir, y que dará como resultado un modelo del sistema que podrá ser implementado en un lenguaje de programación. Este modelo incluirá la identificación y descripción de las diversas entidades del dominio del problema, identificando los objetos del programa y las clases, y se irán relacionando instancias de las mismas cuyas interacciones irán modelando el mundo real, de tal forma que se obtenga la solución al problema del usuario.

El método de análisis empleado implica grandes diferencias en el punto de partida del diseño orientado a objetos. Este análisis previo puede haberse realizado orientado a objetos o no, por lo que las formas de proceder son distintas en los dos casos. Con un análisis orientado a objetos, gran parte de las clases que aparecerán en el diseño ya se han detectado en la fase previa mientras que, en caso contrario, de los conceptos alrededor de los cuales se haya estructurado el análisis (típicamente, funciones y procesos) habrá que inferir las abstracciones (clases) susceptibles de ser utilizadas y refinadas.

3.1 Diseño partiendo de un análisis orientado a objetos

El diseño, en este caso, refina y elabora las clases y relaciones detectadas en el análisis. Se añaden detalles de implementación relativos a:

- ✓ mejoras en la eficiencia mediante la reestructuración de clases.
- ✓ creación de las estructuras de datos internas y los algoritmos de las operaciones de las clases.
- ✓ implementación de las asociaciones y agrupación de clases fuertemente acopladas y relacionadas en paquetes físicos.

El diseño amplía el modelo realizado en el análisis con decisiones de implementación específicas y con clases, atributos, asociaciones y operaciones adicionales requeridas para llegar a la implementación final.

Cada operación detectada en el análisis se debe realizar mediante un algoritmo que, clara y eficientemente, la lleve a cabo.

En la fase de diseño, se reajustan las definiciones de las clases en orden a incrementar la reusabilidad y la abstracción por la herencia.

Una vez concluido el diseño, el paso a la implementación debería ser un proceso sencillo, casi mecánico, puesto que todas las decisiones complicadas deberían haberse realizado en las fases anteriores.

3.2 Diseño partiendo de un análisis no orientado a objetos

Otras metodologías de desarrollo software, utilizadas muy intensamente en los años 90, están basadas en diagramas de flujo de datos. Aun siendo más complicado, es posible partir de un análisis funcional para abordar un diseño orientado a objetos. Lo que se pierde con respecto a una prolongación del análisis orientado a objetos por un diseño orientado a objetos es la suave continuidad (misma notación, mismos conceptos, mismas técnicas) entre ambas fases.

No obstante, hay metodologías orientadas a objetos (OMT o Structured Analysis-Structured Design por ejemplo) que propugnan una fase de análisis con orientación funcional por ser más adecuada al punto de vista del usuario final (que quiere que el sistema 'haga' o "funcione" de una determinada forma).

3.3 Características del diseño orientado a objetos

El diseño orientado a objetos presenta una serie de características consideradas muy deseables en cualquier proceso de diseño. A continuación se exponen algunas de ellas:

1. **Modularidad:** El paradigma de la orientación a objetos aporta un soporte natural a la descomposición de un sistema en módulos, donde las clases se identifican perfectamente con dicho concepto.

Ocultación de implementación (information hiding): La clase aporta la ocultación de detalles de implementación mediante la separación de la interfaz de la clase de su implementación. Lo anterior permite que la misma especificación de interfaz pueda implementarse de diversas maneras de forma totalmente transparente a las otras clases clientes de la misma. Esta separación favorece la abstracción (los detalles quedan ocultos) y permite que las labores de mantenimiento queden ocultas a los usuarios de la clase (siempre y cuando se mantenga la especificación - interfaz- de la misma).

2. **Acoplamiento débil:** Las clases se diseñan como colecciones de datos y operaciones permitidas sobre las mismas. Así pues, los operadores que aparecen en la interfaz son los únicos que pueden acceder o alterar los datos internos a la clase. Esto lleva a que exista un menor acoplamiento entre clases.
3. **Cohesión fuerte:** Una clase es de forma natural un módulo con mucha cohesión puesto que modela claramente una cierta entidad. La propia esencia de la orientación a objetos implica que cualquier función miembro de la interfaz de una clase debe acceder o modificar la estructura de datos contenida en la clase.
4. **Extensibilidad:** Gracias a los mecanismos de herencia y polimorfismo, los diseños orientados a objetos son fácilmente extensibles. La herencia facilita la reutilización de especificaciones preexistentes para la creación de otras nuevas. Gracias al polimorfismo, una clase cliente (que utilice la interfaz) de otra podrá tratar igualmente con objetos pertenecientes a subclases herederas de aquella. Esto es válido tanto para las clases herederas consideradas en el diseño original, como para otras que vayan apareciendo al ir extendiendo el diseño.
5. **Integrable:** El resultado de un diseño orientado a objetos produce resultados que facilitan la integración de trozos individuales en un diseño global.
6. **Reusabilidad:** El paradigma de la orientación a objetos combina técnicas de diseño y características de los lenguajes de programación para facilitar una base sólida para la reutilización de módulos software.

3.3.1 Modelado y Metodologías de Diseño Orientadas a Objetos

Para el desarrollo de software orientado a objetos no basta usar un lenguaje orientado a objetos, sino que también se necesitará realizar un análisis y diseño orientado a objetos, siendo el modelado OO visual la clave para realizar estos últimos. Es decir, que si se usa un lenguaje orientado a objetos sin tener en cuenta las propiedades beneficiosas de herencia, polimorfismo, encapsulamiento, abstracción, etc., tendremos un mal diseño, y es como si el lenguaje no fuera OO.

Desde los inicios del desarrollo de software orientado a objetos han existido diferentes metodologías de modelado. Según Jacobson son:

- ✓ Object-Oriented Design (OOD), Booch.
- ✓ Object Modeling Technique (OMT), Rumbaugh.
- ✓ Object Oriented Analysis (OOA), Coad/Yourdon.
- ✓ Hierarchical Object Oriented Design (HOOD), ESA.
- ✓ Object Oriented Structured Design (OOSD), Wasserman.
- ✓ Object Oriented Systems Analysis (OOSA), Shaler y Mellor.

✓ UML: Unified Modelling Language. Realmente UML no es una metodología (en todo caso se debería considerar RUP) pero se incluye por su importancia como se detalla a continuación. Algunos de estos métodos, en un principio, estaban basados en funciones como es el caso del método de Yourdon, aunque posteriormente han sido adaptados al diseño orientado a objetos. Otros sin embargo, como es el caso del método de Booch, han sido específicamente desarrollados para el Diseño Orientado a Objetos.

El siguiente cuadro resume las distintas metodologías:

SIGLAS	NOMBRE	FASES	DESCRIPCIÓN
GOOD	General Object-Oriented Design	D	Utilizada fundamentalmente para entornos en tiempo real y ADA
HOOD	Hierarchical Object-Oriented Design	D	Se basa en un diseño descendente jerárquico
OOSD	Object-Oriented Structured Design	D	Notación derivada de Booch y de los diagramas de estructura de Constantine que soporta concurrencia
OMT	Object Modelling Technique	A/D/I	Parte de la idea de utilizar los mismos conceptos y notación a lo largo de todo el ciclo de vida
OODLE	Object-Oriented Design Language	D	Notación gráfica independiente del lenguaje para OO de programas, librerías y entornos
OOSE	Object-Oriented Software Engineering	A/D/I	Versión reducida de Objetary ("proceso" industrial para el desarrollo OO de sistemas)
SQMA	Object-Oriented Modelling Approach	A	Extensión de la metodología OOA de Coad y Yourdon, que incorpora algunas ideas de IA y modelización semántica de datos
OOD/ BOOCH	Object-Oriented Design	D/I	Se basa en un descripción muy completa de los sistemas, tanto lógica como física, apoyándose en diagramas
OOA/ OOD	Object-Oriented Analysis & Design	A/D	Elabora 5 "niveles" (clases/objetos, estructuras, sujetos, atributos, servicios) durante el análisis y 4 "componentes" durante el diseño (gestores de tareas, datos e interacción humana)
OOSA	SCHALER y MELLOR	A	Resulta más bien una colección de técnicas estructuradas que de OO

Sin lugar a dudas, el Lenguaje de Modelado Unificado (UML) puso fin a la guerra de metodologías. Actualmente en la industria del desarrollo de software UML se ha convertido en un estándar de facto en el modelado de sistemas orientados a objetos. UML se desarrolla en el tema 88 de Diseño Arquitectónico, y sus técnicas se detallan en varios temas del temario.

3.4 Principios OO

Tradicionalmente, en Orientación a objetos se ha hablado de los principios recogidos bajo los acrónimos de SOLID y GRASP. Muchos de los conceptos que engloban están relacionados con los criterios de control de calidad del diseño Software vistos en el desarrollo estructurado.

3.4.1 Principios SOLID

SOLID es un acrónimo mnemónico introducido por Robert C. Martin (“Conocido como Uncle Bob”) a comienzos de la década del 2000 que representa cinco principios básicos de la programación orientada a objetos y el diseño. Cuando estos principios se aplican en conjunto es más probable que un desarrollador cree un sistema que sea fácil de mantener y ampliar con el tiempo.

Los principios SOLID son guías que pueden ser aplicadas en el desarrollo de software para eliminar código sucio provocando que el programador tenga que refactorizar el código fuente hasta que sea legible y extensible. Es interesante utilizar estos principios con el desarrollo guiado por pruebas o TDD, y forma parte de la estrategia global del desarrollo ágil de software y desarrollo adaptativo de software.

Este Acrónimo hace referencia a los siguientes elementos:

-  **S - Single responsibility principle (SRP)**
-  **O - Open/closed principle (OCP)**
-  **L - Liskov substitution principle (LSP)**
-  **I - Interface segregation principle (ISP)**
-  **D - Dependency inversion principle (DIP)**

A continuación se muestra una descripción más detallada de los cinco principios:

3.4.1.1 Principio de responsabilidad única (Single responsibility principle):

Un objeto solo debería tener una única responsabilidad y que esta debe estar contenida únicamente en la clase. Así:

- ✓ Una clase debería tener sólo una razón para cambiar
- ✓ Cada responsabilidad es el eje del cambio
- ✓ Para contener la propagación del cambio, debemos separar las responsabilidades
- ✓ Si una clase asume más de una responsabilidad, será más sensible al cambio
- ✓ Si una clase asume más de una responsabilidad, las responsabilidades se acoplan

3.4.1.2 Principio de abierto/cerrado (Open/closed principle):

Responde al siguiente concepto, **las “entidades de software... deben estar abiertas para su extensión, pero cerradas para su modificación”**. Se deberían poder añadir campos a la estructura de datos que contiene dicho módulo o nuevas funcionalidades a su comportamiento.

Un módulo queda cerrado si queda utilizable para otros módulos. Esto implica que dicho módulo goza de una descripción estable y bien definida.

Definido por Meyer. Su solución descansaba en el concepto de herencia de la orientación a objetos. En los noventa (Robert C. Martin 1996), el principio de abierto/cerrado se redefinió popularmente para referirse al uso de interfaces abstractas, en las que las implementaciones podían cambiar, incluso podría haber múltiples implementaciones y ser polimórficamente sustituidas unas por otras.

En 2001, Craig Larman relacionó el principio de abierto/cerrado con el patrón de Alistair Cockburn, denominado *Variaciones Protegidas*, y con la discusión de David Parnas sobre *ocultación de la información*.

3.4.1.3 Principio de sustitución de Liskov (Liskov substitution principle):

Responde al siguiente concepto, “**los objetos de un programa deberían ser reemplazables por instancias de sus subtipos sin alterar el correcto funcionamiento del programa**”.

El Principio de Sustitución de Liskov (LSP) es una definición particular de una relación de subtipificación, llamada tipificación (fuerte) del comportamiento, que fue introducido inicialmente por Barbara Liskov en una conferencia magistral en 1987 titulada “*La Abstracción de Datos y Jerarquía*.”

El concepto de comportamiento de subtipos es una extensión de la lógica de Hoare, que tiene una cierta semejanza con el *diseño por contrato* de Bertrand Meyer, ya que considera la interacción de subtipos como pre y postcondiciones.

3.4.1.4 Principio de segregación de la interfaz (Interface segregation principle):

Responde al siguiente concepto, “**muchas interfaces cliente específicas son mejores que una interfaz de propósito general**”. Los clientes de un programa dado sólo deberían conocer de éste aquellos métodos que realmente usan, y no aquellos que no necesitan usar. A este tipo de interfaces reducidas se les llama “interfaces de rol”.

El ISP fue concebido para mantener a un sistema desacoplado respecto a los sistemas de los que depende, y así resulte más fácil refactorizarlo, modificarlo y redespugarlo. El ISP es similar al Principio de Alta Cohesión de GRASP, o al analizado en el análisis estructurado.

3.4.1.5 Principio de inversión de la dependencia (Dependency inversion principle):

Responde al siguiente concepto, “Se debe depender de abstracciones y no de implementaciones”. En la medida que el flujo de programación se complica, saltando de componente a componente, aparece un nuevo problema: la dependencia (acoplamiento) entre los componentes.

El problema de la dependencia se empieza a considerar lo suficientemente importante como para definir nuevos conceptos en el diseño :

- ✚ **Inversión de control (IoC).** Por ejemplo utilizando un framework externo como Spring.
- ✚ **Inyección de Dependencias (DI),** que es una forma de inversión de control.

La inyección de dependencias (en inglés Dependency Injection, DI) es un método/patrón de diseño orientado a objetos, en el que se suministran objetos a una clase en lugar de ser la propia clase quien cree el objeto. El término fue acuñado por primera vez por Martin Fowler.

3.4.2 Principios GRASP

GRASP son patrones generales de software para asignación de responsabilidades, es el acrónimo de “object-oriented design **General Responsibility Assignment Software Patterns**”. Más que patrones propiamente dichos, son una serie de “buenas prácticas” de aplicación recomendable en el diseño de software:

- ✚ **Experto en información:** Principio básico de asignación de responsabilidades. Nos indica, por ejemplo, que la responsabilidad de la creación de un objeto o la implementación de un método, debe recaer sobre la clase que conoce toda la información necesaria para crearlo.
- ✚ **Creador (bajo acoplamiento, mayor claridad, encapsulación y reutilización):** El patrón creador nos ayuda a identificar quién debe ser el responsable de la creación (o instanciación) de nuevos objetos o clases. La nueva instancia deberá ser creada por la clase que:
 - ✓ Tiene la información necesaria para realizar la creación del objeto, o
 - ✓ Usa directamente las instancias creadas del objeto, o
 - ✓ Almacena o maneja varias instancias de la clase
 - ✓ Contiene o agrega la clase.
- ✚ **Controlador:** Patrón que sirve como intermediario entre una determinada interfaz y el algoritmo que la implementa, de tal forma que es la que recibe los datos del usuario y la que los envía a las distintas clases según el método llamado.

Este patrón sugiere que la lógica de negocios debe estar separada de la capa de presentación, esto para aumentar la reutilización de código y a la vez tener un mayor control.

Se recomienda dividir los eventos del sistema en el mayor número de controladores para poder aumentar la cohesión y disminuir el acoplamiento.

- ✚ **Alta cohesión y bajo acoplamiento**
- ✚ **Polimorfismo**
- ✚ **Fabricación pura:** La fabricación pura se da en las clases que no representan un ente u objeto real del dominio del problema, sino que se ha creado intencionadamente para disminuir el acoplamiento, aumentar la cohesión y/o potenciar la reutilización del código. Si se abusa → clases función o algoritmo.
- ✚ **Indirección:** El patrón de indirección nos aporta mejorar el bajo acoplamiento entre dos clases asignando la responsabilidad de la mediación entre ellos a un tercer elemento (clase) intermedio.
- ✚ **Variaciones Protegidas:** principio fundamental de protegerse del cambio, de tal forma que todo lo que preveamos en un análisis previo que es susceptible de modificaciones, lo envolvamos en una interfaz, utilizando el polimorfismo para crear varias implementaciones y posibilitar implementaciones futuras, de manera que quede lo menos ligado posible a nuestro sistema, de forma que cuando se produzca la variación, nos repercuta lo mínimo. Forma parte de los patrones GRASP avanzados.

3.5 Programación Orientada a Aspectos

La Programación Orientada a Aspectos o POA (en inglés: aspect-oriented programming) es un paradigma de programación relativamente reciente cuya intención es permitir una adecuada modularización de las aplicaciones y posibilitar una mejor separación de responsabilidades (Obligación o correspondencia de hacer algo) y responsabilidades.

Gracias a la POA se pueden encapsular los diferentes conceptos que componen una aplicación en entidades bien definidas, eliminando las dependencias entre cada uno de los módulos. De esta forma se consigue razonar mejor sobre los conceptos, se elimina la dispersión del código y las implementaciones resultan más comprensibles, adaptables y reusables. Varias tecnologías con nombres diferentes se encaminan a la consecución de los mismos objetivos y así, el término POA es usado para referirse a varias tecnologías relacionadas como los métodos adaptativos, los filtros de composición, la programación orientada a sujetos o la separación multidimensional de competencias.

El principal objetivo de la POA es la separación de las funcionalidades dentro del sistema:

- ✓ Por un lado funcionalidades comunes utilizadas a lo largo de la aplicación.
- ✓ Por otro lado, las funcionalidades propias de cada módulo.

Cada funcionalidad común se encapsulará en una entidad.

3.5.1 Evolución Historica

La programación orientada a objetos (POO) supuso un gran paso en la ingeniería del software, ya que presentaba un modelo de objetos que parecía encajar de manera adecuada con los problemas reales. La cuestión era saber descomponer de la mejor manera el dominio del problema al que nos enfrentáramos, encapsulando cada concepto en lo que se dicen llamar objetos y haciéndoles interactuar entre ellos, habiéndoles dotado de una serie de propiedades. Surgieron así numerosas metodologías para ayudar en tal proceso de descomposición y aparecieron herramientas que incluso automatizaban parte del proceso. Esto no ha cambiado y se sigue haciendo en el proceso de desarrollo del software. Sin embargo, frecuentemente la relación entre la complejidad de la solución y el problema resuelto hace pensar en la necesidad de un nuevo cambio. Así pues, nos encontramos con muchos problemas donde la POO no es suficiente para capturar de una manera clara todas las propiedades y comportamientos de lo que queremos dotar nuestra aplicación. Así mismo, la programación procedural tampoco nos soluciona el problema. Por lo que ante tales problemas, se utiliza la POA.

Los conceptos y tecnologías reunidos bajo el nombre "programación orientada a aspectos" (AOP, por las siglas de Aspect-Oriented Programming; o AOSD, por Aspect-Oriented Software Development) buscan resolver un problema identificado hace tiempo en el desarrollo de software.

Se trata del problema de la separación de competencias (separation of concerns). AOP no es el único intento por solucionar este problema, del que se habla a continuación: hay varias propuestas, muchas de las cuales se agrupan (junto con AOP) en el campo de estudio denominado ASoC (Advanced Separation of Concerns).

La programación orientada a aspectos (POA) es una nueva metodología de programación que aspira a soportar la separación de competencias para los aspectos antes mencionados. Es decir, que intenta separar los componentes y los aspectos unos de otros, proporcionando mecanismos que hagan posible abstraerlos y componerlos para formar todo el sistema. En definitiva, lo que se persigue es implementar una aplicación de forma eficiente y fácil de entender.

POA es un desarrollo que sigue al paradigma de la orientación a objetos, y como tal, soporta la descomposición orientada a objetos, además de la procedimental y la descomposición funcional. Pero, a pesar de esto, POA no se puede considerar como una extensión de la POO, ya que puede utilizarse con los diferentes estilos de programación ya mencionados.

3.5.2 Principio Separación de Incumbencias

El principio de separación de incumbencias o competencias (Concerns en Inglés) fue identificado en la década de 1970, plantea que un problema dado involucra varias competencias que deben ser identificadas y separadas. Las incumbencias son los diferentes temas o asuntos de los que es necesario ocuparse para resolver el problema. Una de ellas es la función específica que debe realizar una aplicación, pero también surgen otras como por ejemplo distribución, persistencia, replicación, sincronización, etc. Separando las incumbencias, se disminuye la complejidad a la hora de tratarlas y se puede cumplir con requerimientos relacionados con la calidad como adaptabilidad, mantenibilidad, extensibilidad y reusabilidad.

El principio puede aplicarse de distintas maneras. Por ejemplo, separar las fases del proceso de desarrollo puede verse como una separación de actividades de ingeniería en el tiempo y por su objetivo. Definir subsistemas, objetos y componentes son otras formas de poner en práctica el principio de separación de incumbencias. Por eso podemos decir que se trata de un principio rector omnipresente en el proceso de desarrollo de software.

Las técnicas de modelado que se usan en la etapa de diseño de un sistema se basan en partirlo en varios subsistemas que resuelvan parte del problema o correspondan a una parte del dominio sobre el que trata. Estas técnicas sufren en su mayoría la llamada "tiranía de la descomposición dominante" que consiste en guiarse al modelar, implícita o explícitamente, por una visión jerárquica determinada de la organización del sistema. La desventaja de estas particiones es que muchas de las competencias a tener en cuenta para cumplir con los requisitos (en particular, habitualmente, las competencias no funcionales) no suelen adaptarse bien a esa descomposición, como veremos más adelante.

Las construcciones provistas por los lenguajes de programación, que fueron creados para implementar los modelos generados por las técnicas de diseño existentes, reproducen las jerarquías y, por lo tanto, comparten el defecto explicado en el párrafo anterior. En el paradigma de programación imperativa, la descomposición consiste en identificar procedimientos que resuelvan parte del problema, y la jerarquía se da en el árbol de ejecución, según el cual los procedimientos se invocan unos a otros. En el caso de la programación orientada a objetos, la jerarquía generada en la etapa de diseño suele plasmarse en las relaciones de herencia o de composición entre objetos. Por ejemplo, algunos patrones de diseño de uso habitual como observador (observer), visitante (visitor) y mediador (mediator) exhiben estos problemas, ya que para aplicarlos es necesario adaptar a ellos más de una clase.

El problema aparece cuando una competencia afecta a distintas partes del sistema que no aparecen relacionadas en la jerarquía. En ese caso, la única solución suele ser escribir código repetido que resuelva esa competencia para cada subsistema. En inglés este problema se conoce como **scattered code**. Otro problema que puede aparecer, es que un mismo módulo implemente múltiples comportamientos o aspectos del sistema de forma simultánea. En inglés este problema se conoce como **tangled code**, que podríamos traducir como código enmarañado. El hecho es que

hay ciertas decisiones de diseño que son difíciles de capturar, debido a que determinados problemas no se pueden encapsular claramente de igual forma que los que habitualmente se resuelven con funciones u objetos.

3.5.3 Fundamentos de la Programación Orientada a Aspectos

Para tener un programa orientado a aspectos necesitamos definir los siguientes elementos:

- ✓ Un lenguaje para definir la funcionalidad básica. Este lenguaje se conoce como lenguaje base. Suele ser un lenguaje de propósito general, tal como C++ o Java. En general, se podrían utilizar también lenguajes no imperativos.
- ✓ El lenguaje de aspectos define la forma de los aspectos, por ejemplo, los aspectos de AspectJ se programan de forma muy parecida a las clases.
- ✓ El **tejedor** se encargará de combinar los lenguajes. El proceso de mezcla se puede retrasar para hacerse en tiempo de ejecución, o hacerse en tiempo de compilación.

3.5.3.1 Conceptos Básicos

- ✓ **Aspecto:** (en inglés **Aspect**) es una funcionalidad transversal (**cross-cutting**) que se va a implementar de forma modular y separada del resto del sistema. El ejemplo más común y simple de un aspecto es el logging (registro de sucesos) dentro del sistema, ya que necesariamente afecta a todas las partes del sistema que generan un suceso.
- ✓ **Punto de Cruce o de Unión** (en inglés **Join point**): es un punto de ejecución dentro del sistema donde un aspecto puede ser conectado, como una llamada a un método, el lanzamiento de una excepción o la modificación de un campo. El código del aspecto será insertado en el flujo de ejecución de la aplicación para añadir su funcionalidad.
- ✓ **Consejo** (en inglés **Advice**): es la implementación del aspecto, es decir, contiene el código que implementa la nueva funcionalidad. Se insertan en la aplicación en los Puntos de Cruce.
- ✓ **Puntos de Corte** (en inglés: **Pointcut**): define los Consejos que se aplicarán a cada Punto de Cruce. Se especifica mediante Expresiones Regulares o mediante patrones de nombres (de clases, métodos o campos), e incluso dinámicamente en tiempo de ejecución según el valor de ciertos parámetros.
- ✓ **Introducción** (en inglés **Introduction**): permite añadir métodos o atributos a clases ya existentes. Un ejemplo en el que resultaría útil es la creación de un Consejo de Auditoría que mantenga la fecha de la última modificación de un objeto, mediante una variable y un método `setUltimaModificacion(fecha)`, que podrían ser introducidos en todas las clases (o sólo en algunas) para proporcionarles esta nueva funcionalidad.
- ✓ **Destinatario** (en inglés **Target**): es la clase aconsejada, la clase que es objeto de un consejo. Sin AOP, esta clase debería contener su lógica, además de la lógica del aspecto.
- ✓ **Resultante** (en inglés **Proxy**): es el objeto creado después de aplicar el Consejo al Objeto Destinatario. El resto de la aplicación únicamente tendrá que soportar al Objeto Destinatario (pre-AOP) y no al Objeto Resultante (post-AOP).
- ✓ **Tejido** (en inglés **Weaving**): es el proceso de aplicar Aspectos a los Objetos Destinatarios para crear los nuevos Objetos resultantes en los Puntos de Cruce especificados. Este proceso puede ocurrir a lo largo del ciclo de vida del Objeto destinatario, distinguiéndose:
 - ❖ **Aspectos en Tiempo de Compilación**, que necesita un compilador especial.
 - ❖ **Aspectos en Tiempo de Carga**, los Aspectos se implementan cuando el Objeto Destinatario es cargado. Requiere un ClassLoader especial.
 - ❖ **Aspectos en Tiempo de Ejecución**.

3.5.3.2 Características de POA

De la consecución de estos objetivos se pueden obtener las siguientes ventajas:

- ✓ Un código menos enmarañado, más natural y más reducido.
- ✓ Una mayor facilidad para razonar sobre las materias, ya que están separadas y tienen una dependencia mínima.
- ✓ Más facilidad para depurar y hacer modificaciones en el código.

- ✓ Se consigue que un conjunto grande de modificaciones en la definición de una materia tenga un impacto mínimo en las otras.
- ✓ Se tiene un código más reusable y que se puede acoplar y desacoplar cuando sea necesario.

3.6 Diseño por Capas

El diseño por capas es una forma de lograr separación e independencia, características fundamentales para el diseño arquitectónico porque, entre otras cualidades, permiten localizar cambios y facilitan el desarrollo y el mantenimiento.

De manera abstracta este diseño se ilustra en la figura siguiente. La funcionalidad del sistema está organizada en capas separadas, y cada una se apoya sólo en las facilidades y los servicios ofrecidos por la capa inmediatamente debajo de ella.

El diseño por capas no implica distribución de recursos Hardware, pudiendo estar todas las capas representadas corriendo sobre un mismo servidor o bien sobre servidores distintos.



Figura 17.- Arquitectura por capas (N niveles de abstracción)

Este enfoque en capas soporta el desarrollo incremental de sistemas. Conforme se desarrolla una capa, algunos de los servicios proporcionados por esta capa deben quedar a disposición de los usuarios. La arquitectura también es cambiante y portátil. En tanto su interfaz no varíe, una capa puede sustituirse por otra equivalente. Más aún, cuando las interfaces de capa cambian o se agregan nuevas facilidades a una capa, sólo resulta afectada la capa adyacente. A medida que los sistemas en capas localizan dependencias de máquina en capas más internas, se facilita el ofrecimiento de implementaciones multiplataforma de un sistema de aplicación. Sólo las capas más internas dependientes de la máquina deben reimplantarse para considerar las facilidades de un sistema operativo o base de datos diferentes.

Deben tenerse en cuenta siempre a la hora de elegir la arquitectura de un sistema que:

- ✓ No existe problema en el diseño de sistemas que no pueda ser resuelto agregando un nivel de indireccionamiento, es decir una capa adicional de abstracción.
- ✓ No existe problema de rendimiento que no pueda ser resuelto mediante la eliminación de un nivel de indireccionamiento.

El problema en general es encontrar el punto de equilibrio ideal entre ambas aproximaciones. Conviene recordar las métricas de calidad vistas en el apartado 2, Criterios de Evaluación de la Calidad del Diseño.

Los sistemas han ido evolucionando desde una Arquitectura completamente centralizada en la que la capa de presentación, lógica de aplicación y gestión de recursos se construye como una entidad monolítica. Los usuario/programas acceden al sistema a través de terminales de pantalla, pero lo que se muestra y cómo aparece es controlado por el servidor y se podría decir que las relaciones entre usuarios y sistemas son mediante "llamadas de terminales tontas" o con una capacidad de proceso nula o muy limitada.

Ésta era la arquitectura típica de las aplicaciones de mainframe, ofreciendo varias ventajas:

- ✓ flujo de control sin decisiones de contexto (todo sucede dentro del sistema),
- ✓ todo está centralizado; gestión y control de los recursos es más fácil,

- ✓ diseño altamente optimizado al eliminar o no existir la separación entre las capas.

3.6.1 Arquitectura de 2 capas

Al hacerse los ordenadores más potentes, la capa de presentación se mueve hacia el cliente delegándole tareas y por tanto requiriendo capacidad de proceso. Esto tiene varias ventajas:

- ✓ Los clientes son independientes entre sí: se puede tener varias capas de presentación dependiendo de lo que cada cliente quiere hacer.
- ✓ Se puede aprovechar la potencia de cálculo en la máquina cliente teniendo capas de presentación más sofisticadas. Esto también ahorra recursos de la máquina servidor.
- ✓ Se introduce el concepto de API (Application Program Interface). Una interfaz para invocar el sistema desde el exterior.
- ✓ El administrador de recursos sólo tiene un cliente, la lógica de la aplicación. Esto ayuda mucho con el rendimiento ya que no hay conexiones y sesiones para mantener.

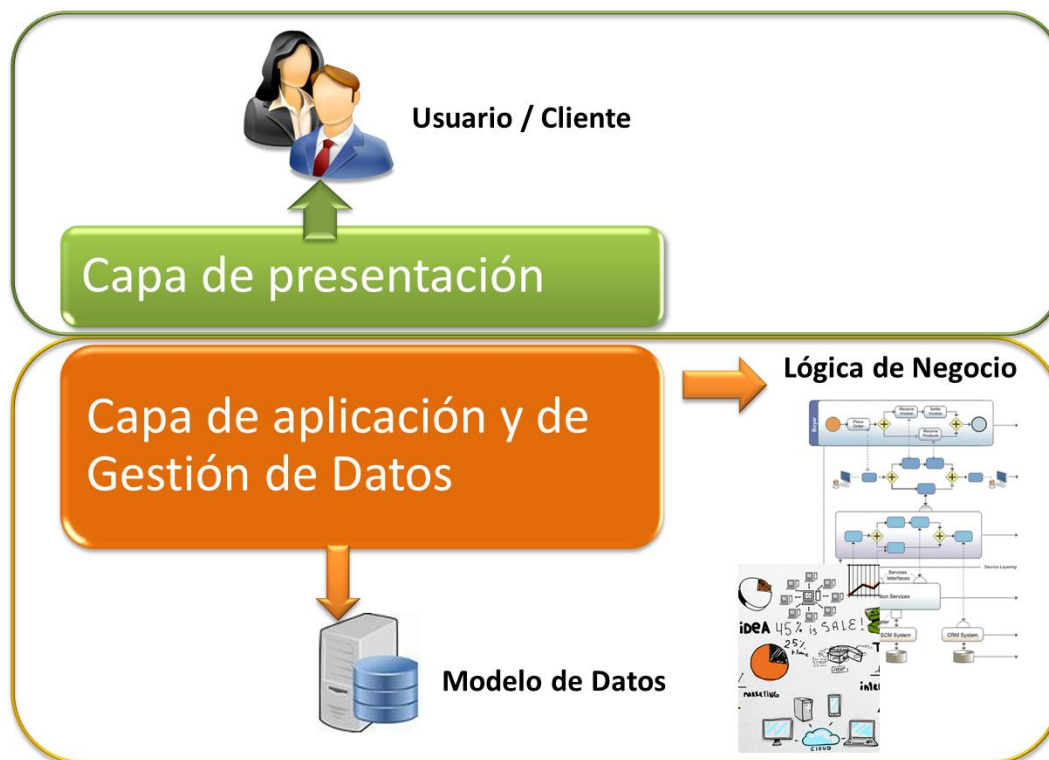


Figura 18 .- Arquitectura a 2 niveles

3.6.2 Middleware

El Middleware es un nivel de indirección entre los clientes y las demás capas del sistema.

Se introduce una capa adicional de la lógica de negocio que abarca todos los sistemas subyacentes.

De esta manera, un sistema middleware:

- ✓ simplifica el diseño de los clientes mediante la reducción del número de interfaces,
- ✓ proporciona un acceso transparente a los sistemas subyacentes,
- ✓ actúa como plataforma para la funcionalidad inter-sistema y la lógica de aplicación de alto nivel.
- ✓ se encarga de localizar los recursos, el acceso a ellos, y la recolección de resultados.

Podría considerarse que un sistema basado en middleware es una arquitectura de 3 capas. Esto es un poco simplista, pero es correcto conceptualmente ya que los sistemas subyacentes pueden ser tratados como cajas negras.

3.6.3 Sistema de 3 capas o multicapa

Posteriormente se introducen los sistemas de tres niveles, en el que existen tres capas que están completamente separadas. Tradicionalmente estas capas suelen ser la de Presentación, Lógica de Negocio y Gestión de datos que se muestra en la figura siguiente.

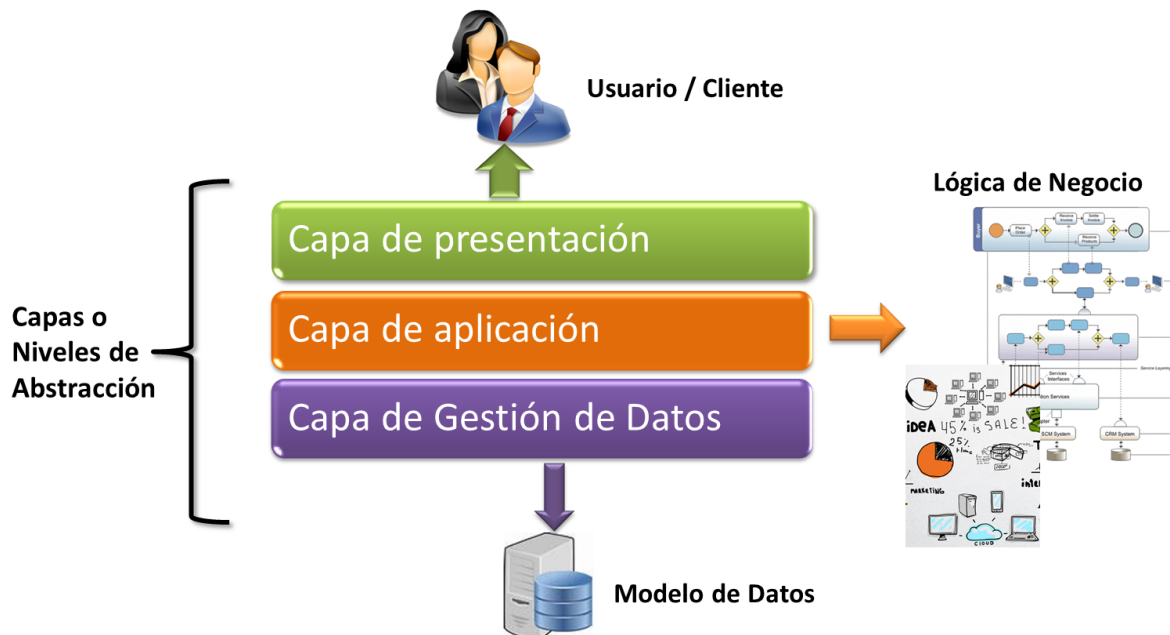


Figura 19.- Arquitectura por capas (3 niveles de abstracción)

Un diseño por capas permite distintas configuraciones y relaciones entre ellas permitiendo:

- ✓ Soporte para múltiples clientes
- ✓ Estrategias de distribución y replicación de datos
- ✓ Lógica de aplicación separada

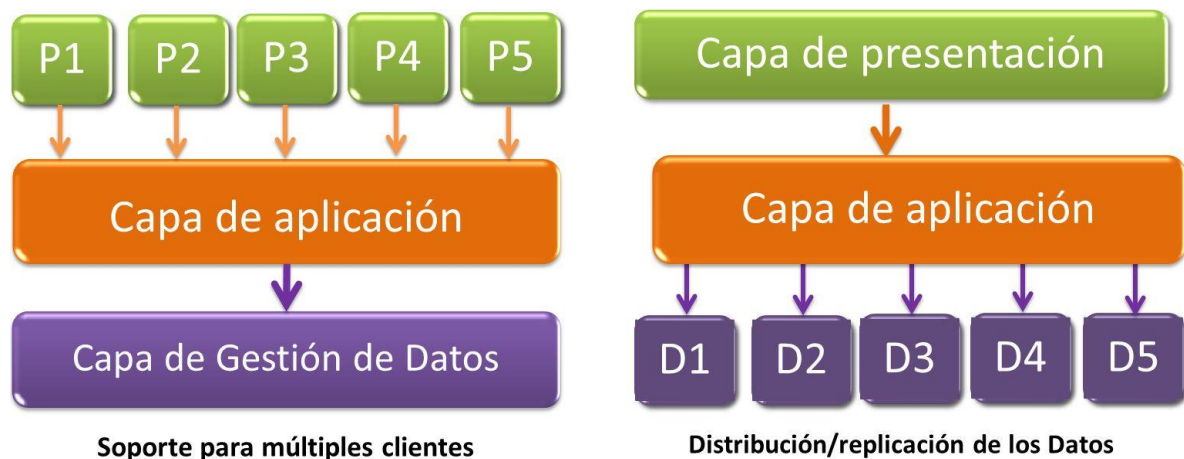


Figura 20.- Configuraciones Multicapa para soportar Múltiples clientes o estrategias de Replicación de datos

Cada cuadro de la figura anterior y de la siguiente representa una parte del sistema. Cada flecha es una conexión entre dos partes del sistema.

Cuanto más cajas, más modular es el sistema, más oportunidades para la distribución y el paralelismo. Esto permite encapsulación, diseño basado en componentes y mejorar la capacidad de reutilización.

A más cajas y más flechas, más sesiones (conexiones) deben mantenerse, más coordinación es necesaria. El sistema se vuelve más complejo de controlar y gestionar como se puede observar en la figura siguiente en el que las combinaciones pueden ser inmanejables.

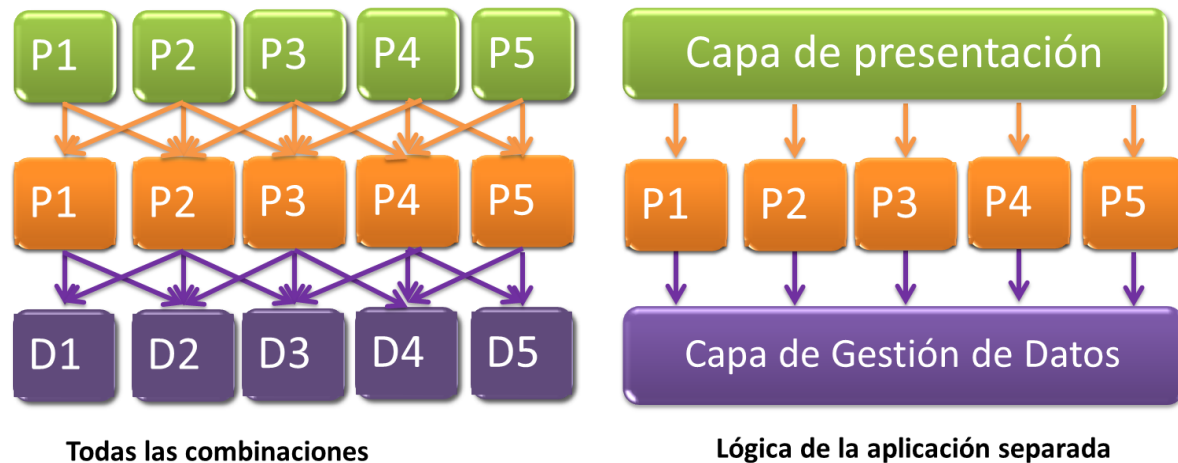


Figura 21.- Configuraciones Multicapa para Lógica de Negocio separada en N aplicaciones y múltiples combinaciones

A más cajas, mayor será el número de decisiones de contexto y pasos intermedios que pasar antes de llegar a los datos. El rendimiento, generalmente, se resiente considerablemente. El diseño, desde el punto de vista de control y relaciones entre capas, como se ha indicado, es más difícil de implementar. Los diseñadores o arquitectos de sistemas tratan de equilibrar la capacidad de los equipos implicados y las ventajas y desventajas de las diferentes arquitecturas. La importancia radica en que permite jugar con el ancho de banda de la red y con la capacidad de proceso de los componentes hardware del sistema para repartir entre ellos la carga de proceso de las lógicas de la aplicación.

Los sistemas de 3 niveles (ampliable a N) en general tienen las mismas ventajas que un sistema de middleware y también sus desventajas. En la práctica, las cosas no son tan simples como parecen... Hay varias capas ocultas que no son necesariamente triviales, por ejemplo los wrappers, parseadores, etc. Se enumeran a continuación de una manera más detallada que en el caso anterior.

3.6.3.1 Ventajas

- ✓ Reduce el tráfico de información en la red por lo que mejora el rendimiento de los sistemas (especialmente respecto a la estructura en dos capas).
- ✓ Brinda una mayor flexibilidad de desarrollo y de elección de plataformas sobre la cual montar las aplicaciones. Facilita la escalabilidad horizontal y vertical.
- ✓ Se mantiene la independencia entre el código de la aplicación (reglas y conocimiento del negocio) y los datos, mejorando la portabilidad de las aplicaciones.
- ✓ Los lenguajes sobre los cuales se desarrollan las aplicaciones son estándares lo que hace más exportables las aplicaciones entre plataformas.
- ✓ Dado que mejora el rendimiento al optimizar el flujo de información entre componentes, permite construir sistemas críticos de alta fiabilidad.
- ✓ El mismo hecho de localizar las reglas del negocio en su propio ambiente, en vez de distribuirlos en la capa de interfaz de usuario, permite reducir el impacto de hacer mantenimiento, cambios urgentes de última hora o mejoras al sistema.
- ✓ Disminuye el número de usuarios (licencias) conectados a la base de datos.

3.6.3.2 Inconvenientes:

Dependiendo de la elección de los lenguajes de desarrollo, puede presentar mayor complejidad en comparación con la arquitectura Cliente/Servidor de dos niveles.

3.6.4 Arquitectura cliente - servidor

La arquitectura cliente-servidor es un modelo de aplicación distribuida en el que las tareas se reparten entre los proveedores de recursos o servicios, llamados servidores, y los demandantes, llamados clientes. Un cliente realiza peticiones a otro programa, el servidor, quien le da respuesta. Esta idea también se puede aplicar a programas que se ejecutan sobre un mismo sistema aunque es más ventajoso aprovechar la capacidad de un sistema distribuido a través de una red de

ordenadores.

Un ejemplo de aplicación cliente-servidor es la Web, con la clásica separación multicapa en la que el servidor se descompone en diferentes programas que pueden ser ejecutados por diferentes ordenadores aumentando así el grado de distribución del sistema.

En esta arquitectura la capacidad de proceso está repartida entre los clientes y los servidores, aunque son más importantes las ventajas de tipo organizativo debidas a la centralización de la gestión de la información y la separación de responsabilidades, lo que facilita y clarifica el diseño del sistema.

La separación entre cliente y servidor es una separación de tipo lógico, donde el servidor no se ejecuta necesariamente sobre una sola máquina ni es necesariamente un sólo programa. Los tipos específicos de servidores incluyen servidores web, servidores de aplicación, servidores de archivo, servidores de correo, etc. Mientras que los propósitos varían de unos servicios a otros, la arquitectura básica seguirá siendo la misma.

Características

- ✓ **Servicio:** unidad básica de diseño. El servidor los proporciona y el cliente los utiliza.
- ✓ **Recursos compartidos:** Muchos clientes utilizan los mismos servidores y, a través de ellos, comparten tanto recursos lógicos como físicos. El Cliente y el Servidor pueden actuar como una sola entidad y también pueden actuar como entidades separadas, realizando actividades o tareas independientes. Las funciones de Cliente y Servidor pueden estar en plataformas separadas, o en la misma plataforma.
- ✓ **Protocolos asimétricos:** Los clientes inician "conversaciones". Los servidores esperan su establecimiento pasivamente.
- ✓ **Transparencia de localización física de los servidores y clientes:** El cliente no tiene por qué saber dónde se encuentra situado el recurso que desea utilizar.
- ✓ **Independencia de la plataforma HW y SW** que se emplee.
- ✓ **Sistemas débilmente acoplados.** Interacción basada en envío de mensajes.
- ✓ **Encapsulamiento de servicios.** Los detalles de la implementación de un servicio son transparentes al cliente.
- ✓ **Escalabilidad horizontal** (añadir clientes) y **vertical** (ampliar potencia de los servidores).
- ✓ **Integridad:** Datos y programas centralizados en servidores facilitan su integridad y mantenimiento. Los cambios realizados en las plataformas de los Clientes o de los Servidores, ya sean por actualización o por reemplazo tecnológico, se realizan de una manera transparente para el usuario final.
- ✓ En el modelo usual Cliente/Servidor, un servidor, (daemon en la terminología sajona basada en sistemas UNIX/LINUX, traducido como "demonio") se activa y espera las solicitudes de los clientes.

3.6.5 Componentes de una arquitectura cliente - servidor

Un sistema Cliente/Servidor está formado por tres elementos fundamentales sobre los cuales se desarrollan e implantan los sistemas Cliente/Servidor: el proceso cliente que es quien inicia el diálogo, el proceso servidor que pasivamente espera a que lleguen peticiones de servicio y el middleware que corresponde a la interfaz que provee la conectividad entre el cliente y el servidor para poder intercambiar mensajes.

Para entender en forma más ordenada y clara los conceptos y elementos involucrados en esta tecnología se puede aplicar una descomposición o arquitectura por capas o niveles vista anteriormente. Esta descomposición principalmente consiste en separar los elementos estructurales de esta tecnología en función de aspectos más funcionales de la misma:

En este caso al tratarse de aplicaciones distribuidas se utilizarían generalmente los siguientes niveles:

- ✓ Nivel de Presentación: Agrupa a todos los elementos asociados al componente Cliente.
- ✓ Nivel de Aplicación: Agrupa a todos los elementos asociados al componente Servidor.
- ✓ Nivel de comunicación: Agrupa a todos los elementos que hacen posible la comunicación entre los componentes Cliente y servidor.
- ✓ Nivel de base de datos: Agrupa a todas las actividades asociadas al acceso de los datos.

Cliente: Un cliente es todo proceso que reclama servicios de otro. Una definición un poco más elaborada podría ser la siguiente: cliente es el proceso que permite al usuario formular las peticiones y pasarlas al servidor. Se lo conoce con el término front-end.

Éste normalmente maneja todas las funciones relacionadas con la manipulación y despliegue de datos, por lo que están desarrollados sobre plataformas que permiten construir interfaces gráficas de usuario (GUI), además de acceder a los servicios distribuidos en cualquier parte de la red.

Las funciones que lleva a cabo el proceso cliente se resumen en los siguientes puntos:

- ✓ Normalmente interactúa directamente con los usuarios finales mediante una interfaz gráfica de usuario que deberá administrar.
- ✓ Interactuar con el usuario.
- ✓ Es quien inicia solicitudes o peticiones, tienen por tanto un papel activo en la comunicación (dispositivo maestro o amo).
- ✓ Espera y recibe las respuestas del servidor.
- ✓ Por lo general, puede conectarse a varios servidores a la vez.
- ✓ Procesar la lógica de la aplicación y hacer validaciones locales.
- ✓ Generar consultas de bases de datos.
- ✓ Recibir resultados del servidor.
- ✓ Formatear resultados.

La funcionalidad del proceso cliente marca la operativa de la aplicación (flujo de información o lógica de negocio). De este modo el cliente se puede clasificar en:

- ✓ Cliente basado en aplicación de usuario. Si los datos son de baja interacción y están fuertemente relacionados con la actividad de los usuarios de esos clientes.
- ✓ Cliente basado en lógica de negocio. Toma datos suministrados por el usuario y/o la base de datos y efectúa los cálculos necesarios según los requerimientos del usuario.

Servidor: todo proceso que proporciona un servicio a otros. Es el proceso encargado de atender a múltiples clientes que hacen peticiones de algún recurso administrado por él. Al proceso servidor se lo conoce con el término back-end. El servidor normalmente maneja todas las funciones relacionadas con la mayoría de las reglas del negocio y los recursos de datos. Las principales funciones que lleva a cabo el proceso servidor se enumeran a continuación:

- ✓ Al iniciarse esperan a que lleguen las solicitudes de los clientes, desempeñan entonces un papel pasivo en la comunicación (dispositivo esclavo).
- ✓ Tras la recepción de una solicitud, la procesan y luego envían la respuesta al cliente.
- ✓ Por lo general, acepta las conexiones de un gran número de clientes. El número máximo de peticiones estará limitado por las características del servidor, en función del número máximo de hilos concurrentes o sesiones que pueda soportar. Si se requiere mayor capacidad deberá escalarse el sistema. Es un requisito No Funcional tratado en el tema 87 en más profundidad).
- ✓ Procesar las consultas de bases de datos que hacen los clientes.
- ✓ Formatear datos para transmitirlos a los clientes.
- ✓ Procesar la lógica de la aplicación y realizar validaciones a nivel de base de datos.

Puede darse el caso que un servidor actúe a su vez como cliente de otro servidor. Existen numerosos tipos de servidores, cada uno de los cuales da lugar a un tipo de arquitectura Cliente/Servidor diferente.

El término "servidor" se suele utilizar también para designar el hardware, de gran potencia, capacidad y prestaciones, utilizado para albergar servicios que atienden a un gran número de usuarios concurrentes. Desde el punto de vista de la arquitectura cliente/servidor y del procesamiento cooperativo un servidor es un servicio software que atiende las peticiones de procesos software clientes.

Es el receptor de la solicitud enviada por el cliente. Sus características son:

- ✓ Nivel de comunicación: Agrupa a todos los elementos que hacen posible la comunicación

- entre los componentes Cliente y servidor.
- ✓ Nivel de base de datos: Agrupa a todas las actividades asociadas al acceso de los datos.
- ✓ Este modelo de descomposición en niveles, como se verá más adelante, permite introducir más claramente la discusión del desarrollo de aplicaciones en arquitecturas de hardware y software en planos.

Lo normal es que los servicios de un mismo servidor puedan ser utilizados por múltiples clientes distintos. Tanto los programas cliente como los servidores son con frecuencia parte de un programa o aplicación mayores. Se pueden consultar las opciones en el apartado anterior relativo a arquitecturas multicapa.

Middleware: módulo intermedio que actúa como conductor entre sistemas permitiendo a cualquier usuario de sistemas de información comunicarse con varias fuentes de información que se encuentran conectadas por una red. En el caso que nos concierne, es el intermediario entre el cliente y el servidor y se ejecuta en ambas partes.

La utilización del middleware permite desarrollar aplicaciones en arquitectura Cliente/Servidor independizando los servidores y clientes, facilitando la interrelación entre ellos y evitando dependencias de tecnologías propietarias. El concepto de middleware no es un concepto nuevo.

Los primeros “**monitores de teleproceso**” de los grandes sistemas basados en tecnología Cliente/Servidor ya se basaban en él, pero es con el nacimiento de la tecnología basada en sistemas abiertos cuando el concepto de middleware toma su máxima importancia. El middleware se estructura en tres niveles:

- ✓ Protocolo de transporte.
- ✓ Network Operating System (NOS).
- ✓ Protocolo específico del servicio, en el caso de que sea requerido.

Las principales características de un middleware son:

- ✓ Simplifica el proceso de desarrollo de aplicaciones al independizar los entornos propietarios.
- ✓ Permite la interconectividad de los Sistemas de Información del Organismo.
- ✓ Proporciona mayor control del negocio al poder contar con información procedente de distintas plataformas sobre el mismo soporte.
- ✓ Facilita el desarrollo de sistemas complejos con diferentes tecnologías y arquitecturas.

La capa de Middleware suele estar proporcionada por el Sistema Operativo donde se ejecuten tanto el cliente como el servidor, y es transparente para el desarrollo, con lo cual no se profundizará más en él.

3.6.6 Tipos de arquitectura cliente / servidor

Uno de los aspectos claves para entender la tecnología Cliente/Servidor, y por tanto contar con la capacidad de proponer y llevar a cabo soluciones de este tipo, es llegar a conocer la arquitectura de este modelo y los conceptos o ideas asociados al mismo.

Más allá de entender los componentes cliente/middleware/servidor, es preciso analizar ciertas relaciones entre éstos, que pueden definir el tipo de solución que se ajusta mejor a las necesidades del negocio, en función del tipo de uso esperado (estadísticas detalladas por tipo de uso) y las restricciones acerca de los eventos y requisitos de información que se obtuvieron en la etapa de análisis de un determinado proyecto.

De hecho el analista deberá conocer estos eventos o restricciones del proyecto para que a partir de ahí, se puedan hacer las consideraciones y estimaciones de la futura configuración, teniendo en cuenta aspectos como por ejemplo, la oportunidad de la información, tiempo de respuesta, tamaños de registros, tamaño de bases de datos, estimaciones del tráfico de red, distribución geográfica tanto de los procesos como de los datos, etc.

En tal sentido se presenta, en primer lugar, un esquema de clasificación basado en el tamaño de

los componentes en función de los grados de libertad que brinda el modelo Cliente/Servidor para balancear la carga de proceso entre los niveles de presentación, aplicación y base de datos. En segundo lugar se mostrará una clasificación según las funciones que ofrecen.

Dependiendo de qué segmento de las capas de software tenga que soportar la mayor o menor carga de procesamiento, se habla de Fat Client (Thin Server) o Fat server (Thin Client), pudiendo variar las configuraciones según se observa en la figura siguiente, que recoge la clasificación según Gartner que se verá más adelante, siendo el cliente más ligero en la configuración de la izquierda y más pesado en la de la derecha.

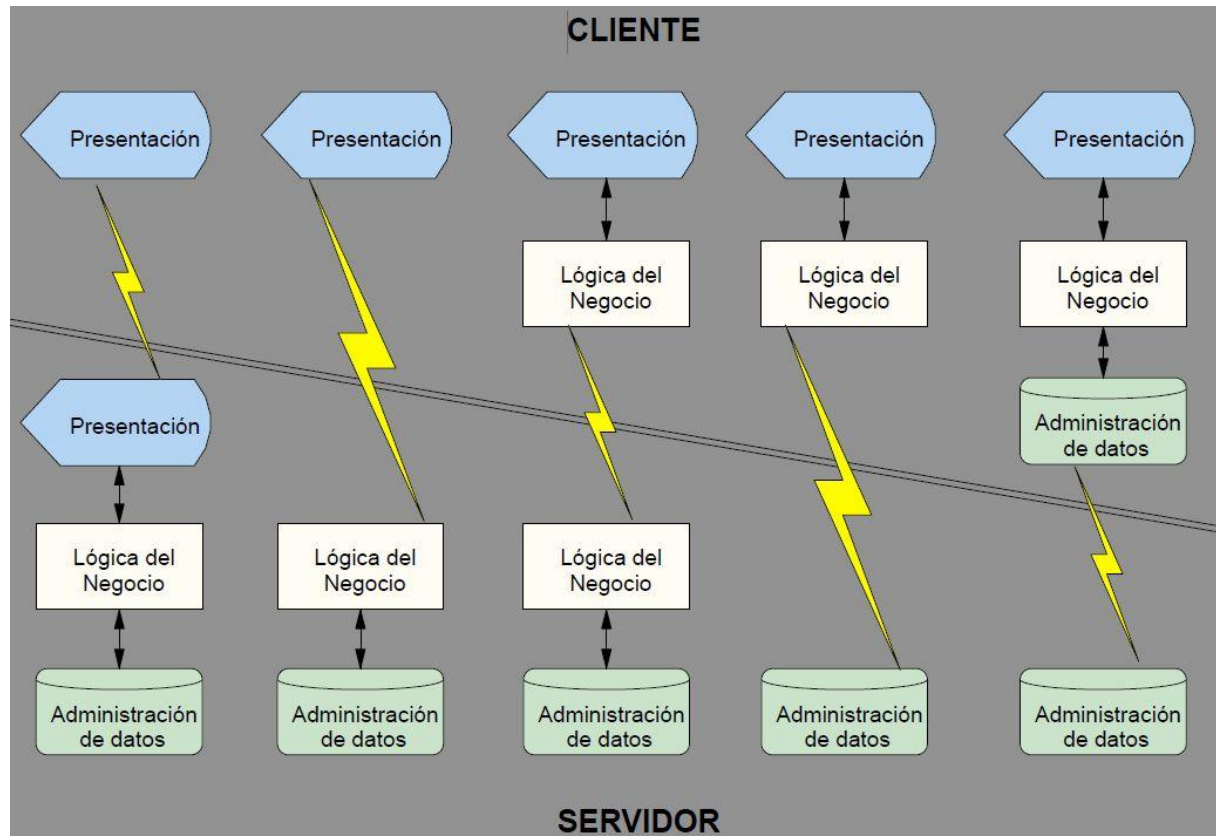


Figura 22.- Configuraciones con distinto grado de integración Cliente-Servidor

Consideraciones de este tipo son importantes en el momento de decidir una plataforma de desarrollo, al mismo tiempo que pueden definir la viabilidad o no de las mismas para enfrentar un cierto número de restricciones impuestas por una problemática a resolver.

3.6.6.1 Fat client (thin server)

En este esquema de arquitectura el peso de la aplicación es ejecutada en el cliente, es decir, el nivel de presentación y el nivel de aplicación corren en un único proceso cliente, y el servidor es relegado a realizar las funciones que provee un administrador de base de datos.

En general este tipo de arquitectura tiene mejor aplicación en sistemas de apoyo de decisiones (DSS: Decision Support System) y sistemas de información ejecutiva (EIS: Executive Information System), y tiene pocas posibilidades de aplicarse en sistemas de misión crítica.

3.6.6.2 Fat server (thin client)

Este es el caso opuesto al anterior, el proceso cliente es restringido a la presentación de la interfaz de usuario, mientras que el peso de la aplicación corre por el lado del servidor de aplicación.

En general este tipo de arquitectura presenta una flexibilidad mayor para desarrollar una gran variedad de aplicaciones, incluyendo los sistemas de misión crítica a través de servidores de transacciones.

3.6.6.3 Separación de funciones

Como se apuntaba anteriormente, las distintas arquitecturas cliente/servidor presentan variaciones acerca de cómo son distribuidas las diferentes funciones de las aplicaciones de sistemas entre el cliente y el servidor, sobre la base de los conceptos de los tres componentes generales de cualquier SI que volvemos a señalar:

- ✓ La lógica de acceso a datos. Funciones que gestionan todas las interacciones entre el SW y los almacenes de datos (archivos, bases de datos, etc.) incluyendo recuperación/consulta, actualización, seguridad y control de concurrencia.
- ✓ La lógica de presentación. Funciones que gestionan la interfaz entre los usuarios del sistema y el SW, incluyendo la visualización e impresión de formas y reportes, y la posibilidad de validar entradas del sistema.
- ✓ La lógica de negocio o lógica de la aplicación. Funciones que transforman entradas en salidas, incluyendo desde simples sumas hasta complejos modelos matemáticos, financieros, científicos, de ingeniería, etc.

Según cómo se distribuyan las funciones correspondientes a estas tres lógicas o funciones de un sistema entre cliente, middleware y servidor (los principales componentes de un sistema con arquitectura distribuida) nos podemos encontrar con los siguientes tipos de arquitectura cliente / servidor que se refleja en la figura anterior: (conforme a la clasificación hecha por el Gartner Group):

- ✓ Presentación Distribuida.
- ✓ Presentación remota.
- ✓ Lógica o procesamiento distribuidos.
- ✓ Acceso a datos remoto.
- ✓ Bases de datos distribuidas.

La importancia de esta clasificación radica en que permite jugar con el ancho de banda de la red y con la capacidad de proceso de los componentes hardware del sistema para repartir entre ellos la carga de proceso de las lógicas de la aplicación.

3.6.6.3.1 Presentación distribuida

El cliente asume parte de las funciones de presentación de la aplicación, ya que siguen existiendo programas en el servidor dedicados a esta tarea. El resto de funciones de la aplicación (negocio, acceso a datos) residen en el servidor.

Esta arquitectura se utiliza para construir emuladores de terminal, aplicaciones de control remoto, front ends gráficos de aplicaciones que residen en un host, etc. Algunos ejemplos de productos que siguen esta filosofía son VLC, Microsoft Terminal Server, Cytrix Metaframe, emulador de host para sistemas operativos modernos como Windows, etc.

La gran ventaja de esta arquitectura es que permite revitalizar sistemas antiguos. Así, las aplicaciones antiguas que funcionaban en entornos host pueden ser empleadas desde modernas estaciones de trabajo Windows o Microsoft (que solamente hacen la función de terminal del host incrustada en un entorno de trabajo de escritorio moderno).

El principal problema es que no se elimina la dependencia del host, no siendo posible la aplicación de los conceptos de downsizing o rightsizing.

3.6.6.3.2 Presentación remota

Toda la lógica de negocio y acceso a datos se ejecuta en el servidor, que en esta ocasión no realiza ninguna función relacionada con la presentación. Todas las funciones de presentación son ejecutadas en el cliente. Un ejemplo de este tipo de aplicaciones son las aplicaciones web, las de los terminales de cajeros automáticos, etc.

La principal ventaja es que la interfaz de usuario se adapta bien a las capacidades del entorno cliente (en la presentación distribuida el servidor tenía que ejecutar funciones dentro de un entorno que podría no ser el más apropiado para el cliente). La principal desventaja es que toda la

información necesaria para la presentación tiene que circular por la red desde el servidor al cliente.

3.6.6.3.3 Lógica o proceso distribuido

La lógica de los procesos se divide entre los distintos componentes del cliente y del servidor. El diseñador de la aplicación debe definir los servicios y las interfaces del sistema de información de forma que los papeles de cliente y servidor sean intercambiables, excepto en el control de los datos que es responsabilidad exclusiva del servidor. En este tipo de situaciones se dice que hay un proceso distribuido o cooperativo.

La principal ventaja de esta arquitectura es que cada uno de los nodos/servidores puede especializarse en un área determinada, de forma que cada proceso se ejecutará en el nodo más apropiado. Además, se pueden reutilizar los sistemas ya existentes (es una especie de antesala del concepto de SOA).

En contrapartida, este tipo de sistemas son más difíciles de diseñar, de mantener y de probar.

3.6.6.3.4 Acceso a datos remoto

El cliente realiza tanto las funciones de presentación como los procesos. Por su parte, el servidor almacena y gestiona los datos que permanecen en una base de datos centralizada. En esta situación se dice que hay una gestión de datos remota.

La principal ventaja de esta arquitectura radica en su sencillez de uso, y su proliferación al ser adaptada por lenguajes de cuarta generación (como Oracle Forms). La desventaja es la latencia de red introducida. Al descargarse toda la lógica de proceso en los aplicativos clientes, estos necesitan descargar los datos necesarios (entradas al proceso) que circulan por la red.

3.6.6.3.5 Bases de datos distribuidas

Este modelo es similar al de Acceso a Datos Remoto, pero además el gestor de base de datos divide sus componentes entre el cliente y el servidor. Las interfaces entre ambos están dentro de las funciones del gestor de datos y, por lo tanto, no tienen impacto en el desarrollo de las aplicaciones. En este nivel se da lo que se conoce como bases de datos distribuidas.

La principal ventaja de este modelo es que facilita el acceso a los datos desde entornos heterogéneos. Los componentes de acceso a datos ubicados en el cliente permiten independizar la base de datos del entorno en el que corren las aplicaciones cliente. Además, permite implementar la "transparencia de ubicación". Este sistema presenta dos importantes inconvenientes:

- ✓ Las bases de datos distribuidas son más difíciles de implementar, y son dependientes del gestor de base de datos (siempre que no existan acuerdos y estándares)
- ✓ La integridad de los datos puede verse comprometida.

3.7 Patrones de Diseño

Un patrón de diseño es una solución "repetible" a un problema "común" de diseño. Para que una solución sea considerada un patrón debe poseer las siguientes características:

- ✚ Haber comprobado su efectividad resolviendo problemas similares en ocasiones anteriores.
- ✚ Ser reutilizable: aplicable a diferentes problemas de diseño en distintas circunstancias.

Los patrones de diseño son la base para la búsqueda de soluciones a problemas comunes en el desarrollo de software y otros ámbitos referentes al diseño de interacción o interfaces.

3.7.1 Origen de los Patrones de diseño

En 1979 el arquitecto Christopher Alexander aportó al mundo de la arquitectura el libro *The Timeless Way of Building*; en él proponía el aprendizaje y uso de una serie de patrones para la construcción de edificios de una mayor calidad.

En palabras de este autor, "Cada patrón describe un problema que ocurre infinidad de veces en nuestro entorno, así como la solución al mismo, de tal modo que podemos utilizar esta solución un millón de veces más adelante sin tener que volver a pensarla otra vez."

Los patrones que Christopher Alexander y sus colegas definieron, publicados en un volumen denominado A Pattern Language, son un intento de formalizar y plasmar de una forma práctica generaciones de conocimiento arquitectónico. Los patrones no son principios abstractos que requieran su redescubrimiento para obtener una aplicación satisfactoria, ni son específicos a una situación particular o cultural; son algo intermedio. Un patrón define una posible solución correcta para un problema de diseño dentro de un contexto dado, describiendo las cualidades invariantes de todas las soluciones.

Más tarde, en 1987, Ward Cunningham y Kent Beck usaron varias ideas de Alexander para desarrollar cinco patrones de interacción hombre-ordenador (HCI) y publicaron un artículo en OOPSLA-87 titulado Using Pattern Languages for OO Programs.

No obstante, no fue hasta principios de la década de 1990 cuando los patrones de diseño tuvieron un gran éxito en el mundo de la informática a partir de la publicación del libro Design Patterns escrito por el grupo Gang of Four (GoF) compuesto por Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides, en el que se recogían 23 patrones de diseño comunes.

3.7.2 Objetivos de los patrones

Los patrones de diseño pretenden:

- ✚ Proporcionar catálogos de elementos reusables en el diseño de sistemas software.
- ✚ Evitar la reiteración en la búsqueda de soluciones a problemas ya conocidos y solucionados anteriormente.
- ✚ Formalizar un vocabulario común entre diseñadores.
- ✚ Estandarizar el modo en que se realiza el diseño.
- ✚ Facilitar el aprendizaje de las nuevas generaciones de diseñadores condensando conocimiento ya existente.

Asimismo, no pretenden:

- ✚ Imponer ciertas alternativas de diseño frente a otras.
- ✚ Eliminar la creatividad inherente al proceso de diseño.

No es obligatorio utilizar los patrones, solo es aconsejable en el caso de tener el mismo problema o similar que soluciona el patrón, siempre teniendo en cuenta que en un caso particular puede no ser aplicable. "Abusar o forzar el uso de los patrones puede ser un error".

3.7.3 Categorías de patrones

Según la escala o nivel de abstracción:

- ✚ Patrones de arquitectura: Aquellos que expresan un esquema organizativo estructural fundamental para sistemas de software.
- ✚ Patrones de diseño: Aquellos que expresan esquemas para definir estructuras de diseño (o sus relaciones) con las que construir sistemas de software.
- ✚ Dialectos: Patrones de bajo nivel específicos para un lenguaje de programación o entorno concreto.

3.7.4 Estructuras o plantillas de patrones

Para describir un patrón se usan plantillas más o menos estandarizadas, de forma que se expresen uniformemente y puedan constituir efectivamente un medio de comunicación uniforme entre diseñadores. Varios autores eminentes en esta área han propuesto plantillas ligeramente distintas, si bien la mayoría definen los mismos conceptos básicos.

La plantilla más común es la utilizada precisamente por el GoF y consta de los apartados descritos por la siguiente tabla.

CATEGORIA	DESCRIPCIÓN
Nombre del patrón	Nombre estándar del patrón por el cual será reconocido en la comunidad (normalmente se expresan en inglés).
Clasificación del patrón	Creacional, estructural o de comportamiento.
Intención	Descripción del problema pretende resolver el patrón.
Nombre Alternativo	Patrón también conocido como - Otros nombres de uso común para el patrón.
Motivación	Escenario de ejemplo para la aplicación del patrón.
Aplicabilidad	Usos comunes y criterios de aplicabilidad del patrón.
Estructura	Diagramas de clases oportunos para describir las clases que intervienen en el patrón.
Participantes	Enumeración y descripción de las entidades abstractas (y sus roles) que participan en el patrón.
Colaboraciones	Explicación de las interrelaciones que se dan entre los participantes.
Consecuencias	Consecuencias positivas y negativas en el diseño derivadas de la aplicación del patrón.
Implementación	Técnicas o comentarios oportunos de cara a la implementación del patrón.
Código de ejemplo	Código fuente ejemplo de implementación del patrón.
Usos conocidos	Ejemplos de sistemas reales que usan el patrón.
Patrones relacionados	Referencias cruzadas con otros patrones.

3.7.5 Relación de principales patrones GoF (Gang Of Four)

En este apartado se recogen los patrones GoF que son una referencia desde el punto de vista del diseño de aplicaciones.

3.7.5.1 Patrones creacionales

-  **Object Pool** (Conjunto de Objetos): (No pertenece a los patrones especificados por GoF) Se obtienen objetos nuevos a través de la clonación. Utilizado cuando el costo de crear una clase es mayor que el de clonarla. Especialmente con objetos muy complejos. Se especifica un tipo de objeto a crear y se utiliza una interfaz del prototipo para crear un nuevo objeto por clonación. El proceso de clonación se inicia instanciando un tipo de objeto de la clase que queremos clonar.

- ✚ **Abstract Factory** (Fábrica abstracta): Permite trabajar con objetos de distintas familias de manera que las familias no se mezclen entre sí y haciendo transparente el tipo de familia concreta que se esté usando.
- ✚ **Builder** (Constructor virtual): Abstrae el proceso de creación de un objeto complejo, centralizando dicho proceso en un único punto.
- ✚ **Factory Method** (Método de fabricación): Centraliza en una clase constructora la creación de objetos de un subtipo de un tipo determinado, ocultando al usuario la casuística para elegir el subtipo que crear.
- ✚ **Prototype** (Prototipo): Crea nuevos objetos clonándolos de una instancia ya existente.
- ✚ **Singleton** (Instancia única): Garantiza la existencia de una única instancia para una clase y la creación de un mecanismo de acceso global a dicha instancia.

3.7.5.2 Patrones estructurales

- ✚ **Adapter** (Adaptador): Adapta una interfaz para que pueda ser utilizada por una clase que de otro modo no podría utilizarla.
- ✚ **Bridge** (Puente): Desacopla una abstracción de su implementación.
- ✚ **Composite** (Objeto compuesto): Permite tratar objetos compuestos como si de uno simple se tratase.
- ✚ **Decorator** (Envoltorio): Añade funcionalidad a una clase dinámicamente.
- ✚ **Facade** (Fachada): Provee de una interfaz unificada simple para acceder a una interfaz o grupo de interfaces de un subsistema.
- ✚ **Flyweight** (Peso ligero): Reduce la redundancia cuando gran cantidad de objetos poseen idéntica información.
- ✚ **Proxy**: Mantiene un representante de un objeto.

3.7.5.3 Patrones de comportamiento

- ✚ **Chain of Responsibility** (Cadena de responsabilidad): Permite establecer la línea que deben llevar los mensajes para que los objetos realicen la tarea indicada.
- ✚ **Command** (Orden): Encapsula una operación en un objeto, permitiendo ejecutar dicha operación sin necesidad de conocer el contenido de la misma.
- ✚ **Interpreter** (Intérprete): Dado un lenguaje, define una gramática para dicho lenguaje, así como las herramientas necesarias para interpretarlo.
- ✚ **Iterator** (Iterador): Permite realizar recorridos sobre objetos compuestos independientemente de la implementación de éstos.
- ✚ **Mediator** (Mediador): Define un objeto que coordine la comunicación entre objetos de distintas clases, pero que funcionan como un conjunto.
- ✚ **Memento** (Recuerdo): Permite volver a estados anteriores del sistema.
- ✚ **Observer** (Observador): Define una dependencia de uno-a-muchos entre objetos, de forma que cuando un objeto cambie de estado se notifique y actualicen automáticamente todos los objetos que dependen de él.
- ✚ **State** (Estado): Permite que un objeto modifique su comportamiento cada vez que cambie su estado interno.
- ✚ **Strategy** (Estrategia): Permite disponer de varios métodos para resolver un problema y elegir cuál utilizar en tiempo de ejecución.
- ✚ **Template Method** (Método plantilla): Define en una operación el esqueleto de un algoritmo, delegando en las subclases algunos de sus pasos, esto permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar su estructura.
- ✚ **Visitor** (Visitante): Permite definir nuevas operaciones sobre una jerarquía de clases sin modificar las clases sobre las que opera.

3.7.6 Antipatrones de diseño software

También es importante reseñar el concepto de "antipatrón de diseño", que con forma semejante a la de un patrón, intenta prevenir contra errores comunes de diseño en el software. La idea de los antipatrones es dar a conocer los problemas que acarrearán ciertos diseños muy frecuentes, para intentar evitar que diferentes sistemas acaben una y otra vez en el mismo callejón sin salida por haber cometido los mismos errores.

Un **antipatrón** de diseño es un patrón de diseño que invariablemente conduce a una mala solución para un problema. Al documentarse los antipatrones, además de los patrones de diseño, se dan argumentos a los diseñadores de sistemas para no escoger malos caminos, partiendo de documentación disponible en lugar de simplemente la intuición.

El término se origina inspirado en el libro Anti-Patterns (de William Brown, Raphael Malveau, Skip McCormick y Tom Mowbray, junto con la más reciente incorporación de Scott Thomas) y describe los antipatrones como la contrapartida natural al estudio de los patrones de diseño. El estudio formal de errores que se repiten permite reconocer y reconducir los elementos involucrados hacia una mejor solución.

Los antipatrones no se mencionan en el libro original de Design Patterns, puesto que éste es anterior. Se consideran una parte importante de una buena práctica de programación. Es decir, un buen programador procurará evitar los antipatrones siempre que sea posible, lo que requiere su reconocimiento e identificación tan pronto como sea posible.

El concepto de antipatrón se puede aplicar a la ingeniería en general, e incluso a cualquier tarea realizada por el hombre. Aunque no se escucha con frecuencia fuera del campo ingenieril, la noción está ampliamente extendida.

3.7.6.1 Antipatrones Típicos

A continuación se recogen las categorías más habituales de antipatrones y ejemplos de cada una. La lista completa se puede consultar en la bibliografía.

- ✚ **Antipatrones de gestión de proyectos** (Humo y espejos, Software inflado)
- ✚ **Antipatrones generales de diseño de software** (Sistema de cañerías de calefacción, Entrada chapuza, Clase Gorda, chapado de oro)
- ✚ **Antipatrones de diseño orientado a objetos** (Singletonitis, Fallo de clase vacía, Objeto todopoderoso)
- ✚ **Antipatrones de programación** (Ancla del barco, Lógica super-booleana, Código espagueti, Programación por excepción)
- ✚ **Antipatrones metodológicos** (Bala de plata, Martillo de oro , Reinventar la rueda, Reinventar la rueda cuadrada)
- ✚ **Antipatrones de gestión de la configuración** (Conflicto de extensiones, Infierno de dependencias)

4 Glosario

CASE: Computer Aided Software Engineering

C/S: Cliente/Servidor

CORBA: Common Object Request Broker Architecture

E/R: Entidad Relación

DEC: Diagrama de Estructura de Cuadros de Constantine

DFD: Diagrama de Flujo de datos

DSI: Diseño del Sistema de Información

GOOD: General Object-Oriented Design

HOOD: Hierarchical Object-Oriented Design

OMT: Object Modelling Technique

OO: Orientación a Objetos

OOA/OOD: Object-Oriented Analysis & Design

OOD: Diseño orientado a objetos

OOSD: Object-Oriented Structured Design

OOSE: Object-Oriented Software Engineering

OODLE: Object-Oriented Design Language

OSF: Open Source Foundation

RFP: Request For Proposal/Solicitud de propuesta

RUP: Rational Unified Process

SGBDR: Sistema de Gestión de Base de Datos Relacional

SOLID: Single responsibility, Open-closed, Liskov substitution, Interface segregation and Dependency inversion

TAD: Tipos Abstractos de Datos

UML: Unified Modelling Language

Conclusiones

En el tema se han presentado distintas técnicas de diseño de sistemas partiendo del análisis estructurado, introduciendo los Principios Generales del Diseño Estructurado de extensión a otros paradigmas de desarrollo como el Orientado a Objetos o el Orientado a Aspectos que combina características de los dos anteriores.

Igualmente se han referenciado los criterios de calidad y buenas prácticas que deben respetarse y tenerse en cuenta a la hora de realizar cualquier diseño SW. Posteriormente se han revisado los conceptos de diseño por capas haciendo un énfasis especial en la arquitectura Cliente/Servidor y finalmente se han repasado los conceptos relativos a patrones y antipatrones de diseño tan importantes en la construcción de sistemas de manera eficiente.

PREGUNTAS DE TEST

- 1) **De las siguientes afirmaciones en relación con el diseño del software, indique cuál de ellas es verdadera :**
 - a) Una característica de un buen diseño es que el acoplamiento entre módulos sea lo más bajo posible.
 - b) Cuantas más conexiones existan entre dos módulos, más bajo es el acoplamiento entre los mismos.
 - c) De los distintos tipos de acoplamiento, el acoplamiento por contenido es el que garantiza la mejor calidad de diseño.
 - d) Son ciertas todas las premisas anteriores.

- 2) **El Análisis de Transacción :**
 - a) Es una modalidad dentro de las técnicas de análisis orientadas al flujo de datos.
 - b) Es una estrategia de diseño para pasar de los Diagramas de Flujo de Datos a los Diagramas de Estructura de Cuadros, que se aplica cuando el DFD toma una forma en la que un dato determina la elección de uno o más flujos de información.
 - c) Es una técnica para la evaluación y el refinamiento del diseño.
 - d) Son falsas todas las premisas anteriores.

- 3) **Dentro de la Fase de Diseño se pueden considerar las siguientes partes :**
 - a) Diseño funcional, diseño orgánico, diseño técnico y diseño detallado.
 - b) Diseño arquitectónico, diseño de datos y diseño procedimental.
 - c) Diseño conceptual y diseño detallado.
 - d) Diseño estructurado, diseño de datos, diseño de procesos y diseño técnico.

- 4) **La cohesión :**
 - a) Es una medida que permite evaluar la calidad de la estructura de los diseños software.
 - b) Es una medida de la relación funcional de los elementos de un módulo.
 - c) El tipo más alto de cohesión es la cohesión procedimental.
 - d) Son correctas las respuestas a) y b).

- 5) **Respecto a la técnica del Diagrama de Estructura de Cuadros, se puede afirmar que :**
 - a) Los elementos principales que contiene son: los procesos, los módulos, las conexiones entre los procesos y los módulos y las comunicaciones entre ambos.
 - b) Las conexiones entre los módulos se efectúan a través de los datos y los "flags".
 - c) Un módulo se puede definir como aquella parte de código que se puede llamar.
 - d) Sólo admite representar la llamada de un módulo como máximo a otro módulo.

- 6) **Los principios del diseño estructurado son :**
 - a) Descomposición, jerarquía e independencia.
 - b) Partición, abstracción, proyección y representación de la estructura modular.
 - c) Independencia, acoplamiento y cohesión.
 - d) Partición, acoplamiento, cohesión y representación de la estructura modular del sistema.

- 7) **En el diseño estructurado, el grado de interdependencia entre los módulos se denomina :**
 - a) Cohesión modular.
 - b) Conexión lógica.
 - c) Comunicación.
 - d) Acoplamiento.

- 8) **En el diseño estructurado, el acoplamiento simple o normal entre dos módulos de software consiste en :**
 - a) Ambos módulos utilizan las mismas estructuras de datos.

- b) Un módulo genera información de control para otro módulo.
- c) Un módulo se comunica con el otro mediante el paso de parámetros.
- d) Son falsas todas las premisas anteriores.

9) A la hora de elegir la arquitectura de un sistema, debe tenerse en cuenta que:

- a) No existe problema de diseño que no pueda ser resuelto mediante la eliminación de un nivel de indireccionamiento.
- b) No existe problema en el rendimiento de sistemas que no pueda ser resuelto agregando un nivel de indireccionamiento, es decir una capa adicional de abstracción.
- c) a y b son ciertas
- d) todas son falsas

10) El diseño por capas permite de manera fácil

- a) Soporte para múltiples clientes
- b) Disponer de una lógica de la aplicación separada
- c) Distribución y replicación de los Datos
- d) Todas las anteriores son ciertas

11) Los tipos de arquitectura cliente / servidor conforme a la clasificación hecha por el Gartner Group son:

- a) Presentación Distribuida, Presentación remota, Lógica distribuida, Acceso a datos remoto y Bases de datos distribuidas.
- b) Presentación Distribuida, Presentación remota y Lógica distribuida
- c) Presentación Distribuida, Presentación remota, Lógica distribuida y Acceso a datos remoto
- d) Presentación remota, Lógica distribuida, Acceso a datos remoto y Bases de datos distribuidas.

12) En relación a la Programación Orientada a Aspectos, señale las ventajas:

- a) Un código menos enmarañado, más natural y más reducido.
- b) Una mayor facilidad para razonar sobre las materias, ya que están separadas y tienen una dependencia mínima.
- c) Más facilidad para depurar y hacer modificaciones en el código.
- d) Todas las anteriores lo son

13) En relación a la Programación Orientada a Aspectos, señale conceptos que no estén relacionados con dicho estilo:

- a) Punto de Cruce
- b) Puntos de Corte
- c) Puntos de Función
- d) Tejido

14) De los siguientes principios SOLID, señale el que no lo sea:

- a) Principio de abierto/cerrado
- b) Principio de asignación de responsabilidades
- c) Principio de sustitución de Liskov
- d) Principio de inversión de la dependencia

15) En función del grado de acoplamiento y de la cohesión, ¿Cuál de los siguientes sistemas sería el más recomendable a efectos de calidad del diseño? :

- a) Un sistema con acoplamiento común y cohesión lógica.
- b) Un sistema con acoplamiento lógico y cohesión común..
- c) Un sistema con acoplamiento de datos y cohesión funcional.
- d) Un sistema con acoplamiento por contenido y cohesión coincidente.

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	A
2	B
3	B
4	D
5	C
6	A
7	D
8	C
9	D
10	D
11	A
12	D
13	C
14	B
15	C

BIBLIOGRAFÍA BÁSICA

1. Metodología Métrica versión 3. Guía de Técnicas y prácticas. Ministerio de Administraciones Públicas 1998.
http://administracionelectronica.gob.es/pae_Home/pae_Documentacion/pae_Metodolog/pae_Metrica_v3.html
2. Pressman, Roger. Ingeniería del Software. Un Enfoque Práctico. Ed. McGraw Hill. 6ª Ed. 2005. ISBN 970-105473-3
3. Sommerville, Ian. Ingeniería del Software. Ed. Addison Wesley. 9ª Ed. 2011. ISBN: 9780137035151
4. Programación Orientada a Aspectos, "proyecto fin de carrera"
http://cybertesis.ubiobio.cl/tesis/2008/almonacid_r/doc/almonacid_r.pdf
5. https://es.wikipedia.org/wiki/Programación_orientada_a_aspectos
6. <http://docs.spring.io/spring/docs/2.5.x/reference/aop.html>
7. https://es.wikipedia.org/wiki/Patrón_de_diseño
8. https://es.wikipedia.org/wiki/Antipatrón_de_diseño

BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

1. Object-Oriented Analysis and Design. G. Booch. Benjamin/Cummings
2. Análisis estructurado moderno.. Prentice. Yourdon, E
3. <http://www.angelfire.com/scifi/jzavalar/apuntes/IngSoftware.html>
4. Distributed Systems: Pearson New International Edition: Principles and Paradigms, Andrew S Tanenbaum - Maarten Van Steen